Carinfo Dashboard

```
<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>CarInfo - In-Vehicle Infotainment (IVI) Cockpit & Scheduling
Studio</title>
    <link rel="stylesheet" href="style.css">
</head>
<body>
    <!-- Top Navigation & Vehicle Cockpit Bar -->
    <header class="navbar">
        <div class="nav-brand">
            <div class="brand-icon">
                <span class="pulse-dot"></span>
                <svg width="24" height="24" viewBox="0 0 24 24"
fill="none" stroke="currentColor" stroke-width="2">
                    <path d="M19 17h2c.6 0 1-.4
1-1v-3c0-.9-.7-1.7-1.5-1.9C18.7 10.6 16 10 16
10s-1.3-1.4-2.2-2.3c-.5-.4-1.1-.7-1.8-.7H5c-.6 0-1.1.4-1.4.9l-1.4 2.9C2.1
11.2 2 11.6 2 12v4c0 .6.4 1 1 1h2"></path>
                    <circle cx="7" cy="17" r="2"></circle>
                    <path d="M9 17h6"></path>
                    <circle cx="17" cy="17" r="2"></circle>
                </svg>
            </div>
            <div>
                <h1 class="brand-title">CarInfo <span
class="badge-accent">IVI Cockpit</span></h1>
                <p class="brand-sub">In-Vehicle Infotainment Performance &
CPU Scheduling Suite</p>
            </div>
        </div>
```

```
        <!-- Vehicle Quick Cluster HUD -->
        <div class="vehicle-hud-strip">
            <div class="hud-item speed-item">
                <span class="hud-label">Vehicle Speed</span>
                <div class="hud-val-wrap">
                    <span class="hud-val" id="hud-speed">64</span>
                    <span class="hud-unit">km/h</span>
                </div>
            </div>
            <div class="hud-item gear-item">
                <span class="hud-label">Gear Shift</span>
                <div class="gear-selector" id="gear-selector">
                    <button class="gear-btn" data-gear="P">P</button>
                    <button class="gear-btn" data-gear="R">R</button>
                    <button class="gear-btn" data-gear="N">N</button>
                    <button class="gear-btn active"
data-gear="D">D</button>
                </div>
            </div>
            <div class="hud-item battery-item">
                <span class="hud-label">Battery Level</span>
                <div class="hud-val-wrap">
                    <span class="hud-val text-emerald"
id="hud-battery">86%</span>
                    <span class="hud-unit">380 km</span>
                </div>
            </div>
            <div class="hud-item cabin-item">
                <span class="hud-label">Climate</span>
                <div class="hud-val-wrap">
                    <span class="hud-val" id="hud-cabin">21.5°C</span>
                    <span class="hud-unit">Dual-Zone</span>
                </div>
            </div>
        </div>

        <div class="nav-controls">
            <div class="status-indicator online">
                <span class="status-dot"></span>
                <span id="backend-status-text">CAN-Bus Active</span>
```

```
            </div>
            <div class="time-display" id="clock-display">00:00:00</div>
            <button class="btn btn-primary" id="btn-sync-db-top">
                <svg width="16" height="16" viewBox="0 0 24 24"
fill="none" stroke="currentColor" stroke-width="2">
                    <path d="M12 2v4M12 18v4M4.93 4.93l2.83 2.83M16.24
16.24l2.83 2.83M2 12h4M18 12h4M4.93 19.07l2.83-2.83M16.24 7.76l2.83-2.83"
/>
                </svg>
                Sync PostgreSQL
            </button>
        </div>
    </header>

    <main class="container">
        <!-- Live System Alerts Notification HUD -->
        <section class="alerts-container" id="alerts-container"></section>

        <!-- Automotive Apps Center: Google Maps & Spotify In-Car Media
-->
        <section class="car-apps-grid">
            <!-- Google Maps In-Car Navigation Widget -->
            <div class="card app-card maps-card">
                <div class="app-header">
                    <div class="app-title-group">
                        <div class="app-icon maps-icon">
                            <svg width="22" height="22" viewBox="0 0 24
24" fill="none" stroke="currentColor" stroke-width="2">
                                <polygon points="3 6 9 3 15 6 21 3 21 18
15 21 9 18 3 21"></polygon>
                                <line x1="9" y1="3" x2="9" y2="18"></line>
                                <line x1="15" y1="6" x2="15"
y2="21"></line>
                            </svg>
                        </div>
                        <div>
                            <div class="flex-center-gap">
                                <h3 class="app-title">Google Maps
Navigation</h3>
```

```
                                <span class="badge-priority
prio-above">ABOVE_NORMAL</span>
                            </div>
                            <p class="app-sub">GPS Turn-by-Turn Routing &
3D Vector Tile Engine</p>
                        </div>
                    </div>
                    <div class="app-perf-pill">
                        <span>CPU: <strong>12.4%</strong></span> &bull;
<span>RAM: <strong>438 MB</strong></span>
                    </div>
                </div>

                <div class="maps-display">
                    <div class="nav-direction-hud">
                        <div class="nav-turn-icon">↱</div>
                        <div class="nav-turn-text">
                            <div class="nav-dist">In 450 meters</div>
                            <div class="nav-street">Turn Right onto Grand
Central Parkway</div>
                        </div>
                        <div class="speed-limit-badge">
                            <span class="limit-label">SPEED</span>
                            <span class="limit-num">80</span>
                        </div>
                    </div>

                    <!-- Simulated GPS Vector Map Canvas/Graphic -->
                    <div class="simulated-map" id="simulated-map">
                        <div class="map-grid-overlay"></div>
                        <div class="map-road main-highway"></div>
                        <div class="map-road cross-street-1"></div>
                        <div class="map-road cross-street-2"></div>
                        <div class="map-route-line"></div>
                        <div class="map-car-cursor">
                            <div class="car-dot"></div>
                            <div class="car-pulse"></div>
                        </div>
                        <div class="map-poi">Downtown Center (14.2
km)</div>
```

```
                    </div>

                    <div class="maps-footer">
                        <div class="eta-details">
                            <span class="eta-time">18 min</span>
                            <span class="eta-stats">14.2 km &bull; 10:48
PM Arrival &bull; Light Traffic</span>
                        </div>
                        <button class="btn btn-sm btn-outline"
id="btn-recalc-route">Recalculate Route</button>
                    </div>
                </div>
            </div>

            <!-- Spotify In-Car Media Player Widget -->
            <div class="card app-card spotify-card">
                <div class="app-header">
                    <div class="app-title-group">
                        <div class="app-icon spotify-icon">
                            <svg width="22" height="22" viewBox="0 0 24
24" fill="none" stroke="currentColor" stroke-width="2">
                                <circle cx="12" cy="12" r="10"></circle>
                                <path d="M8 11.5c2.5-1 5.5-1 8 0"></path>
                                <path d="M7 14.5c3-1.2 7-1.2 10 0"></path>
                                <path d="M9 17.5c2-.8 4-.8 6 0"></path>
                            </svg>
                        </div>
                        <div>
                            <div class="flex-center-gap">
                                <h3 class="app-title">Spotify In-Car
Media</h3>
                                <span class="badge-priority
prio-normal">NORMAL</span>
                            </div>
                            <p class="app-sub">Ogg Vorbis Audio Decoder &
Bluetooth DSP Stream</p>
                        </div>
                    </div>
                    <div class="app-perf-pill">
```

```
                        <span>CPU: <strong>6.2%</strong></span> &bull;
<span>RAM: <strong>194 MB</strong></span>
                    </div>
                </div>

                <div class="spotify-player-body">
                    <div class="track-info-row">
                        <div class="album-artwork">
                            <div class="artwork-glare"></div>
                            <svg width="36" height="36" viewBox="0 0 24
24" fill="currentColor">
                                <path d="M12
3v10.55c-.59-.34-1.27-.55-2-.55-2.21 0-4 1.79-4 4s1.79 4 4 4 4-1.79
4-4V7h4V3h-6z"/>
                            </svg>
                        </div>
                        <div class="track-text">
                            <h4 class="track-title"
id="track-title">Blinding Lights</h4>
                            <p class="track-artist" id="track-artist">The
Weeknd &bull; After Hours</p>
                            <span class="audio-stream-badge">High Fidelity
Lossless &bull; 320 kbps</span>
                        </div>
                    </div>

                    <!-- Audio Frequency Waveform Visualizer -->
                    <div class="audio-waveform-container"
id="audio-waveform">
                        <div class="wave-bar"></div><div
class="wave-bar"></div><div class="wave-bar"></div>
                        <div class="wave-bar"></div><div
class="wave-bar"></div><div class="wave-bar"></div>
                        <div class="wave-bar"></div><div
class="wave-bar"></div><div class="wave-bar"></div>
                        <div class="wave-bar"></div><div
class="wave-bar"></div><div class="wave-bar"></div>
                        <div class="wave-bar"></div><div
class="wave-bar"></div><div class="wave-bar"></div>
                        <div class="wave-bar"></div><div
class="wave-bar"></div><div class="wave-bar"></div>
```

```
                        <div class="wave-bar"></div><div
class="wave-bar"></div><div class="wave-bar"></div>
                    </div>

                    <!-- Media Controls -->
                    <div class="media-controls">
                        <button class="media-btn"
id="btn-prev-track">&#9198;</button>
                        <button class="media-btn play-btn"
id="btn-play-pause">&#9208;</button>
                        <button class="media-btn"
id="btn-next-track">&#9197;</button>
                    </div>

                    <!-- Progress Bar -->
                    <div class="track-progress-bar">
                        <div class="track-progress-fill" style="width:
48%;"></div>
                    </div>
                    <div class="track-timestamps">
                        <span>1:42</span>
                        <span>3:20</span>
                    </div>
                </div>
            </div>

            <!-- Rear-View Safety Camera Monitor Widget -->
            <div class="card app-card camera-card"
id="rear-camera-widget">
                <div class="app-header">
                    <div class="app-title-group">
                        <div class="app-icon camera-icon">
                            <svg width="22" height="22" viewBox="0 0 24
24" fill="none" stroke="currentColor" stroke-width="2">
                                <path d="M23 19a2 2 0 0 1-2 2H3a2 2 0 0
1-2-2V8a2 2 0 0 1 2-2h4l2-3h6l2 3h4a2 2 0 0 1 2 2z"></path>
                                <circle cx="12" cy="13" r="4"></circle>
                            </svg>
                        </div>
                        <div>
```

```
                            <div class="flex-center-gap">
                                <h3 class="app-title">Rear Safety
Camera</h3>
                                <span class="badge-priority prio-realtime"
id="camera-prio-badge">REALTIME</span>
                            </div>
                            <p class="app-sub">60 FPS Ultra-HD Trajectory
Overlay (Safety-Critical)</p>
                        </div>
                    </div>
                    <div class="app-perf-pill">
                        <span id="camera-status-label">Standby (Gear
D)</span>
                    </div>
                </div>

                <div class="camera-view-container" id="camera-view">
                    <div class="camera-overlay-guidelines">
                        <div class="guideline-curve left-guide"></div>
                        <div class="guideline-curve right-guide"></div>
                        <div class="guideline-dist dist-red"></div>
                        <div class="guideline-dist dist-yellow"></div>
                        <div class="guideline-dist dist-green"></div>
                    </div>
                    <div class="camera-message" id="camera-message">
                        <span>Gear in Drive (D): Camera sleeping to
preserve CPU</span>
                        <button class="btn btn-sm btn-outline"
id="btn-trigger-reverse">Simulate Shift to Reverse (R)</button>
                    </div>
                </div>
            </div>
        </section>

        <!-- KPI Metric Cards Grid: IVI SoC Telemetry -->
        <section class="kpi-grid">
            <!-- CPU Load Card -->
            <div class="card kpi-card">
                <div class="kpi-header">
                    <span class="kpi-title">IVI SoC Processor Load</span>
```

```
                    <span class="kpi-badge cpu-badge"
id="cpu-peak-badge">Peak: 0%</span>
                </div>
                <div class="kpi-body">
                    <div class="kpi-value-container">
                        <span class="kpi-number"
id="cpu-percent-val">0.0</span>
                        <span class="kpi-unit">%</span>
                    </div>
                    <div class="gauge-bar-bg">
                        <div class="gauge-bar-fill cpu-fill" id="cpu-bar"
style="width: 0%;"></div>
                    </div>
                    <div class="kpi-meta">
                        <span>Automotive Telemetry Core</span>
                        <span id="cpu-state-label"
class="state-normal">Normal</span>
                    </div>
                </div>
            </div>

            <!-- RAM Usage Card -->
            <div class="card kpi-card">
                <div class="kpi-header">
                    <span class="kpi-title">Infotainment Memory
(RAM)</span>
                    <span class="kpi-badge ram-badge"
id="ram-percent-badge">0%</span>
                </div>
                <div class="kpi-body">
                    <div class="kpi-value-container">
                        <span class="kpi-number"
id="ram-used-val">0</span>
                        <span class="kpi-unit">/ <span
id="ram-total-val">0</span> MB</span>
                    </div>
                    <div class="gauge-bar-bg">
                        <div class="gauge-bar-fill ram-fill" id="ram-bar"
style="width: 0%;"></div>
                    </div>
```

```
                    <div class="kpi-meta">
                        <span>LPDDR4x Automotive Grade</span>
                        <span id="ram-free-val">0 MB free</span>
                    </div>
                </div>
            </div>

            <!-- CAN-Bus Network Load Card -->
            <div class="card kpi-card">
                <div class="kpi-header">
                    <span class="kpi-title">CAN-Bus Network Load</span>
                    <span class="kpi-badge proc-badge"
id="can-status-badge">26.8%</span>
                </div>
                <div class="kpi-body">
                    <div class="kpi-value-container">
                        <span class="kpi-number"
id="can-bus-val">26.8</span>
                        <span class="kpi-unit">% bus load</span>
                    </div>
                    <div class="gauge-bar-bg">
                        <div class="gauge-bar-fill proc-fill" id="can-bar"
style="width: 27%;"></div>
                    </div>
                    <div class="kpi-meta">
                        <span>CAN-FD 5 Mbps Network</span>
                        <span id="can-frames-sec">1,420 frames/sec</span>
                    </div>
                </div>
            </div>

            <!-- Active IVI Services Card -->
            <div class="card kpi-card">
                <div class="kpi-header">
                    <span class="kpi-title">Active Services &
Uptime</span>
                    <span class="kpi-badge uptime-badge">Online</span>
                </div>
                <div class="kpi-body">
                    <div class="kpi-value-container">
```

```
                        <span class="kpi-number uptime-str"
id="uptime-val">00h 00m</span>
                        <span class="kpi-unit">running</span>
                    </div>
                    <div class="gauge-bar-bg">
                        <div class="gauge-bar-fill uptime-fill"
style="width: 100%;"></div>
                    </div>
                    <div class="kpi-meta">
                        <span>Active Tasks: <strong
id="proc-count-val">7</strong></span>
                        <span id="timestamp-val">Updated Live</span>
                    </div>
                </div>
            </div>
        </section>

        <!-- Real-Time Hardware Charts Section -->
        <section class="charts-section">
            <div class="card chart-card">
                <div class="card-header">
                    <div class="card-title-group">
                        <div class="chart-legend-icon cpu-color"></div>
                        <div>
                            <h2 class="card-title">Real-Time IVI SoC
Processor Utilization</h2>
                            <p class="card-subtitle">Continuous 60-second
hardware telemetry trace</p>
                        </div>
                    </div>
                    <div class="chart-stats">
                        <span>Current: <strong
id="chart-cpu-current">0.0%</strong></span>
                        <span>Avg: <strong
id="chart-cpu-avg">0.0%</strong></span>
                    </div>
                </div>
                <div class="canvas-container">
                    <canvas id="cpu-chart" width="600"
height="220"></canvas>
```

```
                </div>
            </div>

            <div class="card chart-card">
                <div class="card-header">
                    <div class="card-title-group">
                        <div class="chart-legend-icon ram-color"></div>
                        <div>
                            <h2 class="card-title">Real-Time Infotainment
Memory Stream</h2>
                            <p class="card-subtitle">Dynamic allocation
across Google Maps, Spotify, and safety services</p>
                        </div>
                    </div>
                    <div class="chart-stats">
                        <span>Current: <strong
id="chart-ram-current">0.0%</strong></span>
                        <span>Used: <strong id="chart-ram-used-mb">0
MB</strong></span>
                    </div>
                </div>
                <div class="canvas-container">
                    <canvas id="ram-chart" width="600"
height="220"></canvas>
                </div>
            </div>
        </section>

        <!-- Automotive Process Explorer & Priority Manager -->
        <section class="process-section">
            <div class="card">
                <div class="card-header flex-between">
                    <div>
                        <h2 class="card-title">Automotive IVI Subsystems &
Priority Manager</h2>
                        <p class="card-subtitle">Inspect infotainment
processes and assign OS execution priorities via
<code>SetPriorityClass</code></p>
                    </div>
                    <div class="table-actions">
```

```
                        <div class="search-box">
                            <svg width="16" height="16" viewBox="0 0 24
24" fill="none" stroke="currentColor" stroke-width="2">
                                <circle cx="11" cy="11" r="8"></circle>
                                <line x1="21" y1="21" x2="16.65"
y2="16.65"></line>
                            </svg>
                            <input type="text" id="proc-search-input"
placeholder="Search service or PID...">
                        </div>
                        <select id="priority-filter" class="form-select">
                            <option value="ALL">All Priority
Classes</option>
                            <option value="REALTIME">REALTIME
(Safety-Critical)</option>
                            <option value="HIGH">HIGH (Vehicle
Network/CAN)</option>
                            <option value="ABOVE_NORMAL">ABOVE_NORMAL
(Maps Navigation)</option>
                            <option value="NORMAL">NORMAL (Spotify &
Media)</option>
                            <option
value="BELOW_NORMAL">BELOW_NORMAL</option>
                            <option value="IDLE">IDLE (Cloud
Diagnostics/OTA)</option>
                        </select>
                        <button class="btn btn-secondary"
id="btn-refresh-procs">Refresh</button>
                    </div>
                </div>

                <div class="table-responsive">
                    <table class="data-table" id="process-table">
                        <thead>
                            <tr>
                                <th style="width: 90px;">PID</th>
                                <th>Subsystem / Process Name</th>
                                <th>Category</th>
                                <th style="width: 160px;">Memory
(RAM)</th>
```

```
                                <th style="width: 90px;">Threads</th>
                                <th style="width: 150px;">Priority
Class</th>
                                <th style="width: 150px; text-align:
right;">Action</th>
                            </tr>
                        </thead>
                        <tbody id="process-table-body">
                            <tr>
                                <td colspan="7"
class="loading-cell">Loading real-time process list from C++
engine...</td>
                            </tr>
                        </tbody>
                    </table>
                </div>
            </div>
        </section>

        <!-- Automotive CPU Process Scheduling Studio -->
        <section class="scheduler-section" id="scheduling-studio">
            <div class="card">
                <div class="card-header flex-between">
                    <div>
                        <h2 class="card-title">Automotive CPU Scheduling
Simulation Studio</h2>
                        <p class="card-subtitle">Evaluate how Round Robin
(RR) and Priority Scheduling prevent infotainment UI lag and guarantee
safety</p>
                    </div>
                    <div class="scheduler-mode-buttons">
                        <button class="btn btn-tab active"
id="btn-mode-rr">Round Robin (RR)</button>
                        <button class="btn btn-tab"
id="btn-mode-priority">Priority Scheduling</button>
                    </div>
                </div>

                <!-- Automotive Presets Bar -->
                <div class="automotive-presets-bar">
```

```
                    <span class="preset-label">Automotive Test
Scenarios:</span>
                    <button class="btn btn-sm btn-outline"
id="preset-reverse">
                        🚗 Scenario 1: Reverse Gear Safety Preemption
(Camera interrupts Spotify)
                    </button>
                    <button class="btn btn-sm btn-outline"
id="preset-multiapp">
                        🎵 Scenario 2: Multi-App Round Robin (Google Maps
+ Spotify + HVAC)
                    </button>
                    <button class="btn btn-sm btn-outline"
id="preset-can">
                        ⚠️ Scenario 3: CAN-Bus Critical Sensor Interrupt
                    </button>
                </div>

                <!-- Simulation Configuration Bar -->
                <div class="scheduler-controls-bar">
                    <div class="control-group" id="rr-quantum-group">
                        <label for="time-quantum-input">Time Quantum (q
):</label>
                        <input type="number" id="time-quantum-input"
min="1" max="20" value="3" class="form-input-num">
                    </div>
                    <div class="control-group" id="priority-preempt-group"
style="display: none;">
                        <label class="checkbox-label">
                            <input type="checkbox"
id="preemptive-checkbox" checked>
                            <span>Preemptive Mode (High priority
camera/CAN interrupts ongoing app)</span>
                        </label>
                    </div>
                    <div class="control-actions">
                        <button class="btn btn-outline"
id="btn-add-sim-proc">+ Add Custom Service</button>
                        <button class="btn btn-success"
id="btn-run-simulation">
```

```
                            <svg width="14" height="14" viewBox="0 0 24
24" fill="none" stroke="currentColor" stroke-width="2">
                                <polygon points="5 3 19 12 5 21 5
3"></polygon>
                            </svg>
                            Run Simulation
                        </button>
                    </div>
                </div>

                <!-- Process Input Table for Scheduling -->
                <div class="table-responsive sim-input-wrapper">
                    <table class="data-table" id="sim-input-table">
                        <thead>
                            <tr>
                                <th style="width: 90px;">PID</th>
                                <th>Automotive Subsystem</th>
                                <th style="width: 140px;">Arrival Time (AT
)</th>
                                <th style="width: 140px;">Burst Time (BT
)</th>
                                <th style="width: 140px;">Priority
(1=Safety Max)</th>
                                <th style="width: 80px; text-align:
center;">Remove</th>
                            </tr>
                        </thead>
                        <tbody id="sim-input-table-body"></tbody>
                    </table>
                </div>

                <!-- Simulation Output: Gantt Chart -->
                <div class="sim-results-container"
id="sim-results-container">
                    <div class="gantt-section">
                        <div class="gantt-header">
                            <h3 class="subsection-title">Automotive Gantt
Chart Execution Timeline</h3>
                            <span class="badge-accent"
id="gantt-algo-badge">Round Robin (q=3)</span>
```

```
                        </div>
                        <div class="gantt-wrapper">
                            <div class="gantt-chart"
id="gantt-chart"></div>
                            <div class="gantt-timeline-ticks"
id="gantt-ticks"></div>
                        </div>
                    </div>

                    <!-- Scheduling Metrics Summary Scoreboard -->
                    <div class="metrics-scoreboard">
                        <div class="score-card">
                            <div class="score-label">Avg Turnaround Time (
TAT)</div>
                            <div class="score-value"
id="score-avg-tat">0.00</div>
                            <div class="score-sub">TAT = CT − AT</div>
                        </div>
                        <div class="score-card">
                            <div class="score-label">Avg Waiting Time (WT
)</div>
                            <div class="score-value"
id="score-avg-wt">0.00</div>
                            <div class="score-sub">WT = TAT − BT</div>
                        </div>
                        <div class="score-card">
                            <div class="score-label">IVI CPU
Utilization</div>
                            <div class="score-value text-emerald"
id="score-cpu-util">100%</div>
                            <div class="score-sub">Active Processor
Duty</div>
                        </div>
                        <div class="score-card">
                            <div class="score-label">System
Throughput</div>
                            <div class="score-value"
id="score-throughput">0.00</div>
                            <div class="score-sub">Tasks / Time
Slice</div>
```

```
                    </div>
                </div>

                <!-- Calculated Process Output Table -->
                <div class="table-responsive sim-output-table-wrap">
                    <table class="data-table" id="sim-output-table">
                        <thead>
                            <tr>
                                <th>PID</th>
                                <th>Subsystem</th>
                                <th>Arrival (AT)</th>
                                <th>Burst (BT)</th>
                                <th>Priority</th>
                                <th>Completion (CT)</th>
                                <th>Turnaround (TAT)</th>
                                <th>Waiting (WT)</th>
                                <th>Response (RT)</th>
                            </tr>
                        </thead>
                        <tbody id="sim-output-table-body"></tbody>
                    </table>
                </div>
            </div>
        </div>
    </section>

    <!-- PostgreSQL Database Sync & Schema Viewer -->
    <section class="database-section">
        <div class="card">
            <div class="card-header flex-between">
                <div>
                    <h2 class="card-title">PostgreSQL Automotive
Telemetry Database</h2>
                    <p class="card-subtitle">Automated logging of
vehicle speed, battery, IVI app states, priority changes, and
simulations</p>
                </div>
                <div class="db-badges">
```

```
                        <span class="badge-db"
id="db-status-badge">PostgreSQL: Connected</span>
                        <button class="btn btn-primary"
id="btn-sync-db-bottom">Commit Telemetry Snapshot</button>
                    </div>
                </div>

                <div class="db-details-grid">
                    <div class="db-info-panel">
                        <h4 class="info-panel-title">Active Database
Tables</h4>
                        <ul class="db-table-list">
                            <li>
                                <span class="table-dot"></span>
                                <strong>vehicle_telemetry</strong>
                                <p class="table-desc">Vehicle speed
(km/h), RPM, battery %, cabin temperature, gear (P/R/N/D).</p>
                            </li>
                            <li>
                                <span class="table-dot"></span>
                                <strong>system_metrics</strong>
                                <p class="table-desc">IVI SoC CPU load,
RAM used, total memory, process counts.</p>
                            </li>
                            <li>
                                <span class="table-dot"></span>
                                <strong>process_snapshots</strong>
                                <p class="table-desc">Active footprints
for Google Maps, Spotify, Rear Camera, and CAN-Bus.</p>
                            </li>
                            <li>
                                <span class="table-dot"></span>
                                <strong>process_priority_logs</strong>
                                <p class="table-desc">Audit logs of
priority elevation (e.g. Camera shift to REALTIME).</p>
                            </li>
                            <li>
                                <span class="table-dot"></span>
                                <strong>scheduling_simulations</strong>
```

```
                                <p class="table-desc">Historical runs of
Round Robin & Priority Scheduling.</p>
                            </li>
                        </ul>
                        <div class="db-actions-row">
                            <button class="btn btn-outline btn-sm"
id="btn-view-sql">View Generated SQL</button>
                            <button class="btn btn-outline btn-sm"
id="btn-download-sql">Download .SQL Dump</button>
                        </div>
                    </div>
                    <div class="db-sql-preview">
                        <div class="code-header">
                            <span>Live SQL Export Buffer</span>
                            <span class="copy-badge"
id="btn-copy-sql">Copy</span>
                        </div>
                        <pre class="code-block"><code
id="sql-preview-code">-- Generating live automotive SQL
buffer...</code></pre>
                    </div>
                </div>
            </div>
        </section>
    </main>

    <!-- Priority Change Modal -->
    <div class="modal-overlay" id="priority-modal">
        <div class="modal-card">
            <div class="modal-header">
                <div>
                    <h3 class="modal-title">Modify Automotive Process
Priority</h3>
                    <p class="modal-subtitle"
id="modal-proc-info">Subsystem: google_maps_nav.bin (PID: 1042)</p>
                </div>
                <button class="modal-close"
id="btn-close-modal">&times;</button>
            </div>
            <div class="modal-body">
```

```
                <p class="modal-desc">Assign an OS priority class to this
infotainment subsystem via <code>SetPriorityClass</code>:</p>

                <div class="priority-options">
                    <label class="priority-option">
                        <input type="radio" name="priority-choice"
value="REALTIME">
                        <div class="prio-info">
                            <span class="prio-name">REALTIME
(Safety-Critical)</span>
                            <span class="prio-desc">Dedicated to Backup
Camera and ADAS warnings. Preempts all other apps.</span>
                        </div>
                    </label>
                    <label class="priority-option">
                        <input type="radio" name="priority-choice"
value="HIGH">
                        <div class="prio-info">
                            <span class="prio-name">HIGH (CAN-Bus /
Cluster)</span>
                            <span class="prio-desc">Vehicle network
message arbitration and digital speedometer sync.</span>
                        </div>
                    </label>
                    <label class="priority-option">
                        <input type="radio" name="priority-choice"
value="ABOVE_NORMAL">
                        <div class="prio-info">
                            <span class="prio-name">ABOVE_NORMAL (Google
Maps Navigation)</span>
                            <span class="prio-desc">Smooth GPS vector
rendering and live turn calculations.</span>
                        </div>
                    </label>
                    <label class="priority-option">
                        <input type="radio" name="priority-choice"
value="NORMAL" checked>
                        <div class="prio-info">
                            <span class="prio-name">NORMAL (Spotify &
Media)</span>
```

```
                            <span class="prio-desc">Standard user-facing
infotainment apps sharing CPU via Round Robin.</span>
                        </div>
                    </label>
                    <label class="priority-option">
                        <input type="radio" name="priority-choice"
value="BELOW_NORMAL">
                        <div class="prio-info">
                            <span class="prio-name">BELOW_NORMAL
(Non-critical)</span>
                            <span class="prio-desc">Lower priority
applications.</span>
                        </div>
                    </label>
                    <label class="priority-option">
                        <input type="radio" name="priority-choice"
value="IDLE">
                        <div class="prio-info">
                            <span class="prio-name">IDLE (Cloud
Diagnostics & OTA Sync)</span>
                            <span class="prio-desc">Only runs when the
vehicle head unit processor is completely idle.</span>
                        </div>
                    </label>
                </div>

                <div class="modal-feedback" id="modal-feedback"
style="display: none;"></div>
            </div>
            <div class="modal-footer">
                <button class="btn btn-secondary"
id="btn-cancel-modal">Cancel</button>
                <button class="btn btn-primary"
id="btn-confirm-priority">Apply Priority Class</button>
            </div>
        </div>
    </div>

    <!-- Footer -->
    <footer class="footer">
```

```
        <p>CarInfo In-Vehicle Infotainment (IVI) Suite &bull; MinGW GCC
C++14 Engine &bull; PostgreSQL 12+ Bridge</p>
    </footer>

    <script src="app.js"></script>
</body>
</html>
```

Styling the webpage

```
/*
================================================================================
   CarInfo In-Vehicle Infotainment (IVI) Cockpit - Modern Dark HUD
Stylesheet

================================================================================
*/

:root {
    --bg-main: #06080e;
    --bg-card: rgba(14, 20, 34, 0.82);
    --bg-card-hover: rgba(20, 28, 48, 0.9);
    --border-color: rgba(255, 255, 255, 0.08);
    --border-glow: rgba(0, 242, 254, 0.25);

    --text-primary: #f8fafc;
    --text-secondary: #94a3b8;
    --text-muted: #64748b;

    --cyan-accent: #026065;
    --cyan-glow: rgba(0, 242, 254, 0.18);
    --spotify-green: #1db954;
    --spotify-glow: rgba(29, 185, 84, 0.2);
    --maps-blue: #aae95c;
    --maps-glow: rgba(66, 133, 244, 0.2);
    --purple-accent: #a855f7;
    --emerald-accent: #10b981;
    --amber-accent: #f59e0b;
    --rose-accent: #f43f5e;

    --radius-sm: 6px;
    --radius-md: 10px;
    --radius-lg: 16px;
    --radius-xl: 22px;

    --transition: all 0.2s cubic-bezier(0.4, 0, 0.2, 1);
```

```
}

* {
    box-sizing: border-box;
    margin: 0;
    padding: 0;
}

body {
    background-color: var(--bg-main);
    background-image:
        radial-gradient(circle at 10% 10%, rgba(0, 242, 254, 0.05) 0%,
transparent 40%),
        radial-gradient(circle at 90% 90%, rgba(29, 185, 84, 0.04) 0%,
transparent 40%),
        radial-gradient(circle at 50% 50%, rgba(66, 133, 244, 0.03) 0%,
transparent 60%);
    color: var(--text-primary);
    font-family: 'Segoe UI', Arial, sans-serif;
    line-height: 1.5;
    min-height: 100vh;
    padding-bottom: 40px;
    overflow-x: hidden;
}

/* Custom Scrollbars */
::-webkit-scrollbar { width: 6px; height: 6px; }
::-webkit-scrollbar-track { background: rgba(0, 0, 0, 0.2); }
::-webkit-scrollbar-thumb { background: rgba(255, 255, 255, 0.15);
border-radius: 3px; }
::-webkit-scrollbar-thumb:hover { background: rgba(0, 242, 254, 0.4); }

code, pre, .mono { font-family: Consolas, 'Courier New', monospace; }

/* Top Navbar */
.navbar {
    display: flex;
    justify-content: space-between;
    align-items: center;
    padding: 14px 28px;
```

```
    background: rgba(8, 12, 20, 0.88);
    backdrop-filter: blur(16px);
    border-bottom: 1px solid var(--border-color);
    position: sticky;
    top: 0;
    z-index: 100;
    gap: 16px;
    flex-wrap: wrap;
}

.nav-brand {
    display: flex;
    align-items: center;
    gap: 12px;
}

.brand-icon {
    width: 44px;
    height: 44px;
    background: linear-gradient(135deg, rgba(0, 242, 254, 0.2), rgba(66,
133, 244, 0.2));
    border: 1px solid rgba(0, 242, 254, 0.4);
    border-radius: var(--radius-md);
    display: flex;
    align-items: center;
    justify-content: center;
    color: var(--cyan-accent);
    position: relative;
    box-shadow: 0 0 20px rgba(0, 242, 254, 0.2);
}

.pulse-dot {
    position: absolute;
    top: -2px;
    right: -2px;
    width: 8px;
    height: 8px;
    background-color: var(--emerald-accent);
    border-radius: 50%;
    box-shadow: 0 0 8px var(--emerald-accent);
```

```
    animation: pulse 2s infinite;
}

@keyframes pulse {
    0% { transform: scale(0.95); opacity: 0.8; }
    50% { transform: scale(1.3); opacity: 1; }
    100% { transform: scale(0.95); opacity: 0.8; }
}

.brand-title {
    font-size: 1.35rem;
    font-weight: 800;
    letter-spacing: -0.5px;
    display: flex;
    align-items: center;
    gap: 8px;
}

.brand-sub {
    font-size: 0.78rem;
    color: var(--text-secondary);
}

.badge-accent {
    font-size: 0.65rem;
    font-weight: 700;
    text-transform: uppercase;
    background: var(--cyan-glow);
    color: var(--cyan-accent);
    border: 1px solid rgba(53, 201, 208, 0.3);
    padding: 2px 8px;
    border-radius: 999px;
    letter-spacing: 0.5px;
}

/* Vehicle Quick Cluster Strip */
.vehicle-hud-strip {
    display: flex;
    align-items: center;
    gap: 20px;
```

```
    background: rgba(0, 0, 0, 0.45);
    padding: 8px 18px;
    border-radius: 999px;
    border: 1px solid var(--border-color);
}

.hud-item {
    display: flex;
    flex-direction: column;
    align-items: center;
}

.hud-label {
    font-size: 0.68rem;
    text-transform: uppercase;
    letter-spacing: 0.6px;
    color: var(--text-muted);
}

.hud-val-wrap {
    display: flex;
    align-items: baseline;
    gap: 4px;
}

.hud-val {
    font-size: 1.25rem;
    font-weight: 700;
    font-family: Consolas, 'Courier New', monospace;
    color: var(--cyan-accent);
}

.hud-unit {
    font-size: 0.72rem;
    color: var(--text-secondary);
}

.gear-selector {
    display: flex;
    gap: 4px;
```

```
    margin-top: 2px;
}

.gear-btn {
    background: rgba(255, 255, 255, 0.06);
    border: 1px solid rgba(255, 255, 255, 0.12);
    color: var(--text-secondary);
    padding: 2px 8px;
    border-radius: 4px;
    font-weight: 700;
    font-size: 0.75rem;
    cursor: pointer;
    transition: var(--transition);
}

.gear-btn:hover { background: rgba(255, 255, 255, 0.14); }

.gear-btn.active {
    background: var(--cyan-accent);
    color: #000;
    border-color: var(--cyan-accent);
    box-shadow: 0 0 10px rgba(0, 242, 254, 0.5);
}

.gear-btn[data-gear="R"].active {
    background: var(--rose-accent);
    color: #fff;
    border-color: var(--rose-accent);
    box-shadow: 0 0 12px rgba(244, 63, 94, 0.6);
}

.nav-controls {
    display: flex;
    align-items: center;
    gap: 16px;
}

.status-indicator {
    display: flex;
    align-items: center;
```

```
    gap: 8px;
    font-size: 0.82rem;
    font-weight: 500;
    background: rgba(255, 255, 255, 0.04);
    padding: 6px 12px;
    border-radius: 999px;
    border: 1px solid var(--border-color);
}

.status-dot {
    width: 8px;
    height: 8px;
    border-radius: 50%;
    background-color: var(--emerald-accent);
    box-shadow: 0 0 10px var(--emerald-accent);
}

.time-display {
    font-family: Consolas, 'Courier New', monospace;
    font-size: 0.92rem;
    color: var(--cyan-accent);
    background: rgba(0, 0, 0, 0.4);
    padding: 6px 12px;
    border-radius: var(--radius-sm);
    border: 1px solid var(--border-color);
}

/* Container */
.container {
    max-width: 1440px;
    margin: 24px auto;
    padding: 0 20px;
    display: flex;
    flex-direction: column;
    gap: 24px;
}

/* Base Card */
.card {
    background: var(--bg-card);
```

```
    backdrop-filter: blur(14px);
    border: 1px solid var(--border-color);
    border-radius: var(--radius-lg);
    padding: 22px;
    box-shadow: 0 8px 32px rgba(0, 0, 0, 0.35);
    transition: var(--transition);
}

.card:hover { border-color: rgba(255, 255, 255, 0.14); }

.card-header.flex-between {
    display: flex;
    justify-content: space-between;
    align-items: center;
    flex-wrap: wrap;
    gap: 16px;
    margin-bottom: 18px;
}

.card-title-group {
    display: flex;
    align-items: center;
    gap: 12px;
}

.card-title { font-size: 1.15rem; font-weight: 600; letter-spacing:
-0.3px; }
.card-subtitle { font-size: 0.82rem; color: var(--text-secondary);
margin-top: 2px; }

/* Buttons */
.btn {
    display: inline-flex;
    align-items: center;
    gap: 8px;
    font-family: inherit;
    font-size: 0.85rem;
    font-weight: 600;
    padding: 8px 16px;
    border-radius: var(--radius-md);
```

```
    cursor: pointer;
    border: 1px solid transparent;
    transition: var(--transition);
}

.btn-primary {
    background: #08b6c9;
    color: #07131a;
    border-color: #08b6c9;
}

.btn-primary:hover {
    background: #079bad;
    border-color: #079bad;
}

.btn-secondary {
    background: rgba(255, 255, 255, 0.08);
    color: var(--text-primary);
    border-color: var(--border-color);
}

.btn-secondary:hover { background: rgba(255, 255, 255, 0.14); }

.btn-outline {
    background: transparent;
    color: var(--text-primary);
    border-color: rgba(255, 255, 255, 0.18);
}

.btn-outline:hover {
    background: rgba(255, 255, 255, 0.08);
    border-color: var(--cyan-accent);
}

.btn-success {
    background: linear-gradient(135deg, #059669, #10b981);
    color: #042014;
    box-shadow: 0 0 15px rgba(16, 185, 129, 0.3);
}
```

```css
.btn-success:hover {
    transform: translateY(-1px);
    box-shadow: 0 0 25px rgba(16, 185, 129, 0.5);
}

.btn-sm { padding: 6px 12px; font-size: 0.78rem; }

.btn-tab {
    background: rgba(255, 255, 255, 0.05);
    color: var(--text-secondary);
    border: 1px solid var(--border-color);
    border-radius: var(--radius-sm);
    padding: 6px 16px;
}

.btn-tab.active {
    background: var(--cyan-glow);
    color: var(--cyan-accent);
    border-color: var(--cyan-accent);
}

/*
================================================================================
   Automotive Apps Center: Google Maps & Spotify Widgets

================================================================================
*/

.car-apps-grid {
    display: grid;
    grid-template-columns: 1.4fr 1fr 1fr;
    gap: 20px;
}

@media (max-width: 1200px) {
    .car-apps-grid { grid-template-columns: 1fr; }
}

.app-card {
```

```
    padding: 20px;
    display: flex;
    flex-direction: column;
    justify-content: space-between;
}

.app-header {
    display: flex;
    justify-content: space-between;
    align-items: center;
    margin-bottom: 16px;
    flex-wrap: wrap;
    gap: 10px;
}

.app-title-group {
    display: flex;
    align-items: center;
    gap: 12px;
}

.app-icon {
    width: 40px;
    height: 40px;
    border-radius: var(--radius-md);
    display: flex;
    align-items: center;
    justify-content: center;
}

.maps-icon {
    background: var(--maps-glow);
    color: var(--maps-blue);
    border: 1px solid rgba(66, 133, 244, 0.4);
}

.spotify-icon {
    background: var(--spotify-glow);
    color: var(--spotify-green);
    border: 1px solid rgba(29, 185, 84, 0.4);
```

```
}

.camera-icon {
    background: rgba(244, 63, 94, 0.15);
    color: var(--rose-accent);
    border: 1px solid rgba(244, 63, 94, 0.4);
}

.app-title {
    font-size: 1.05rem;
    font-weight: 700;
}

.app-sub {
    font-size: 0.76rem;
    color: var(--text-secondary);
}

.flex-center-gap {
    display: flex;
    align-items: center;
    gap: 8px;
}

.badge-priority {
    font-size: 0.65rem;
    font-weight: 700;
    padding: 2px 6px;
    border-radius: 4px;
    font-family: Consolas, 'Courier New', monospace;
}

.prio-realtime { background: rgba(244, 63, 94, 0.2); color: #fda4af;
border: 1px solid rgba(244, 63, 94, 0.4); }
.prio-above { background: rgba(66, 133, 244, 0.2); color: #93c5fd; border:
1px solid rgba(66, 133, 244, 0.4); }
.prio-normal { background: rgba(29, 185, 84, 0.2); color: #86efac; border:
1px solid rgba(29, 185, 84, 0.4); }

.app-perf-pill {
```

```
    font-size: 0.74rem;
    background: rgba(255, 255, 255, 0.05);
    border: 1px solid var(--border-color);
    padding: 4px 10px;
    border-radius: 999px;
    color: var(--text-secondary);
}

/* Google Maps Display */
.maps-display {
    display: flex;
    flex-direction: column;
    gap: 12px;
}

.nav-direction-hud {
    background: #0b1329;
    border: 1px solid rgba(66, 133, 244, 0.3);
    border-radius: var(--radius-md);
    padding: 10px 14px;
    display: flex;
    align-items: center;
    gap: 12px;
}

.nav-turn-icon {
    font-size: 1.5rem;
    color: var(--cyan-accent);
    font-weight: 800;
}

.nav-turn-text {
    flex-grow: 1;
}

.nav-dist {
    font-size: 0.88rem;
    font-weight: 700;
    color: #fff;
}
```

```
.nav-street {
    font-size: 0.74rem;
    color: var(--text-secondary);
}

.speed-limit-badge {
    width: 36px;
    height: 36px;
    border-radius: 50%;
    background: #fff;
    border: 3px solid #dc2626;
    display: flex;
    flex-direction: column;
    align-items: center;
    justify-content: center;
    color: #000;
}

.limit-label { font-size: 0.45rem; font-weight: 800; line-height: 1; }
.limit-num { font-size: 0.78rem; font-weight: 800; line-height: 1; }

.simulated-map {
    position: relative;
    height: 140px;
    background: #090f1e;
    border-radius: var(--radius-md);
    border: 1px solid rgba(255, 255, 255, 0.06);
    overflow: hidden;
}

.map-grid-overlay {
    position: absolute;
    inset: 0;
    background-image:
        linear-gradient(rgba(255, 255, 255, 0.03) 1px, transparent 1px),
        linear-gradient(90deg, rgba(255, 255, 255, 0.03) 1px, transparent
1px);
    background-size: 20px 20px;
}
```

```css
.map-road.main-highway {
    position: absolute;
    top: 50%;
    left: 0;
    width: 100%;
    height: 24px;
    background: #1e293b;
    transform: translateY(-50%) rotate(-8deg);
    border-top: 1px dashed rgba(255, 255, 255, 0.2);
    border-bottom: 1px dashed rgba(255, 255, 255, 0.2);
}

.map-road.cross-street-1 {
    position: absolute;
    top: 0;
    left: 35%;
    width: 16px;
    height: 100%;
    background: #1e293b;
}

.map-road.cross-street-2 {
    position: absolute;
    top: 0;
    left: 75%;
    width: 16px;
    height: 100%;
    background: #1e293b;
}

.map-route-line {
    position: absolute;
    top: 48%;
    left: 15%;
    width: 60%;
    height: 5px;
    background: var(--maps-blue);
    box-shadow: 0 0 10px var(--maps-blue);
    transform: rotate(-8deg);
```

```
    border-radius: 4px;
}

.map-car-cursor {
    position: absolute;
    top: 46%;
    left: 45%;
    transform: translate(-50%, -50%);
}

.car-dot {
    width: 12px;
    height: 12px;
    background: var(--cyan-accent);
    border-radius: 50%;
    border: 2px solid #fff;
    box-shadow: 0 0 10px var(--cyan-accent);
}

.car-pulse {
    position: absolute;
    top: -6px;
    left: -6px;
    width: 24px;
    height: 24px;
    border-radius: 50%;
    border: 2px solid var(--cyan-accent);
    animation: carPulse 1.8s infinite;
}

@keyframes carPulse {
    0% { transform: scale(0.5); opacity: 1; }
    100% { transform: scale(1.6); opacity: 0; }
}

.map-poi {
    position: absolute;
    bottom: 8px;
    right: 12px;
    font-size: 0.7rem;
```

```
    color: var(--text-secondary);
    background: rgba(0, 0, 0, 0.6);
    padding: 2px 8px;
    border-radius: 4px;
}

.maps-footer {
    display: flex;
    justify-content: space-between;
    align-items: center;
}

.eta-time {
    font-size: 1.15rem;
    font-weight: 800;
    color: var(--emerald-accent);
    margin-right: 8px;
}

.eta-stats {
    font-size: 0.74rem;
    color: var(--text-secondary);
}

/* Spotify In-Car Player */
.spotify-player-body {
    display: flex;
    flex-direction: column;
    gap: 12px;
}

.track-info-row {
    display: flex;
    align-items: center;
    gap: 14px;
}

.album-artwork {
    width: 60px;
    height: 60px;
```

```
    background: linear-gradient(135deg, #121212, #282828);
    border-radius: var(--radius-md);
    border: 1px solid rgba(255, 255, 255, 0.1);
    display: flex;
    align-items: center;
    justify-content: center;
    color: var(--spotify-green);
    position: relative;
    overflow: hidden;
    box-shadow: 0 4px 15px rgba(0, 0, 0, 0.4);
}

.track-title { font-size: 0.96rem; font-weight: 700; }
.track-artist { font-size: 0.76rem; color: var(--text-secondary); }
.audio-stream-badge { font-size: 0.68rem; color: var(--spotify-green);
font-weight: 600; }

.audio-waveform-container {
    display: flex;
    align-items: flex-end;
    gap: 3px;
    height: 36px;
    padding: 0 6px;
    background: rgba(0, 0, 0, 0.3);
    border-radius: var(--radius-sm);
}

.wave-bar {
    flex-grow: 1;
    background: var(--spotify-green);
    border-radius: 2px;
    height: 30%;
    animation: waveBounce 1.2s infinite ease-in-out alternate;
}

.wave-bar:nth-child(2n) { animation-delay: 0.2s; height: 60%; }
.wave-bar:nth-child(3n) { animation-delay: 0.4s; height: 80%; }
.wave-bar:nth-child(4n) { animation-delay: 0.1s; height: 45%; }
.wave-bar:nth-child(5n) { animation-delay: 0.5s; height: 95%; }
```

```css
@keyframes waveBounce {
    0% { height: 20%; }
    100% { height: 90%; }
}

.media-controls {
    display: flex;
    align-items: center;
    justify-content: center;
    gap: 16px;
}

.media-btn {
    background: rgba(255, 255, 255, 0.08);
    border: 1px solid var(--border-color);
    color: var(--text-primary);
    width: 34px;
    height: 34px;
    border-radius: 50%;
    cursor: pointer;
    display: flex;
    align-items: center;
    justify-content: center;
    font-size: 0.85rem;
    transition: var(--transition);
}

.media-btn:hover { background: rgba(255, 255, 255, 0.16); }

.media-btn.play-btn {
    width: 44px;
    height: 44px;
    background: var(--spotify-green);
    color: #000;
    border-color: var(--spotify-green);
    font-size: 1.1rem;
    box-shadow: 0 0 15px rgba(29, 185, 84, 0.4);
}

.track-progress-bar {
```

```
    width: 100%;
    height: 4px;
    background: rgba(255, 255, 255, 0.1);
    border-radius: 2px;
    overflow: hidden;
}

.track-progress-fill {
    height: 100%;
    background: var(--spotify-green);
}

.track-timestamps {
    display: flex;
    justify-content: space-between;
    font-size: 0.7rem;
    color: var(--text-muted);
}

/* Rear Safety Camera */
.camera-card {
    transition: var(--transition);
}

.camera-card.reverse-active {
    border-color: var(--rose-accent);
    box-shadow: 0 0 30px rgba(244, 63, 94, 0.35);
}

.camera-view-container {
    height: 200px;
    background: #080c14;
    border-radius: var(--radius-md);
    position: relative;
    overflow: hidden;
    display: flex;
    align-items: center;
    justify-content: center;
    border: 1px solid rgba(255, 255, 255, 0.06);
}
```

```css
.camera-overlay-guidelines {
    position: absolute;
    inset: 0;
    pointer-events: none;
}

.guideline-curve.left-guide {
    position: absolute;
    bottom: 0;
    left: 20%;
    width: 30%;
    height: 80%;
    border-left: 3px solid rgba(255, 255, 255, 0.4);
    transform: skewX(-20deg);
}

.guideline-curve.right-guide {
    position: absolute;
    bottom: 0;
    right: 20%;
    width: 30%;
    height: 80%;
    border-right: 3px solid rgba(255, 255, 255, 0.4);
    transform: skewX(20deg);
}

.guideline-dist {
    position: absolute;
    left: 30%;
    right: 30%;
    height: 2px;
}

.dist-green { bottom: 60%; background: #10b981; }
.dist-yellow { bottom: 40%; background: #f59e0b; }
.dist-red { bottom: 20%; background: #f43f5e; box-shadow: 0 0 8px #f43f5e;
}

.camera-message {
```

```
    z-index: 10;
    text-align: center;
    display: flex;
    flex-direction: column;
    align-items: center;
    gap: 10px;
    font-size: 0.8rem;
    color: var(--text-secondary);
}

/* KPI Grid */
.kpi-grid {
    display: grid;
    grid-template-columns: repeat(auto-fit, minmax(260px, 1fr));
    gap: 20px;
}

.kpi-card {
    padding: 20px;
    display: flex;
    flex-direction: column;
    justify-content: space-between;
}

.kpi-header {
    display: flex;
    justify-content: space-between;
    align-items: center;
    margin-bottom: 12px;
}

.kpi-title {
    font-size: 0.8rem;
    text-transform: uppercase;
    letter-spacing: 0.8px;
    color: var(--text-secondary);
    font-weight: 600;
}

.kpi-badge {
```

```
    font-size: 0.72rem;
    padding: 3px 8px;
    border-radius: 999px;
    font-weight: 600;
}

.cpu-badge { background: var(--cyan-glow); color: var(--cyan-accent);
border: 1px solid rgba(0, 242, 254, 0.3); }
.ram-badge { background: var(--purple-accent); color: #fff; }
.proc-badge { background: var(--spotify-glow); color:
var(--spotify-green); border: 1px solid rgba(29, 185, 84, 0.3); }
.uptime-badge { background: rgba(255, 255, 255, 0.08); color:
var(--text-primary); }

.kpi-value-container {
    display: flex;
    align-items: baseline;
    gap: 6px;
    margin-bottom: 12px;
}

.kpi-number {
    font-size: 2.2rem;
    font-weight: 700;
    font-family: Consolas, 'Courier New', monospace;
    letter-spacing: -1px;
}

.kpi-number.uptime-str { font-size: 1.7rem; }
.kpi-unit { font-size: 0.95rem; color: var(--text-secondary); }

.gauge-bar-bg {
    width: 100%;
    height: 6px;
    background: rgba(255, 255, 255, 0.08);
    border-radius: 999px;
    overflow: hidden;
    margin-bottom: 12px;
}
```

```
.gauge-bar-fill { height: 100%; border-radius: 999px; transition: width
0.4s ease; }
.cpu-fill { background: linear-gradient(90deg, #0284c7,
var(--cyan-accent)); box-shadow: 0 0 10px var(--cyan-accent); }
.ram-fill { background: linear-gradient(90deg, #7c3aed,
var(--purple-accent)); }
.proc-fill { background: linear-gradient(90deg, #059669,
var(--emerald-accent)); }
.uptime-fill { background: rgba(255, 255, 255, 0.4); }

.kpi-meta { display: flex; justify-content: space-between; font-size:
0.75rem; color: var(--text-muted); }

/* Charts */
.charts-section {
    display: grid;
    grid-template-columns: repeat(auto-fit, minmax(480px, 1fr));
    gap: 20px;
}

.chart-card { padding: 20px; }
.chart-legend-icon { width: 12px; height: 12px; border-radius: 3px; }
.cpu-color { background-color: #08b6c9; }
.ram-color { background-color: #9b83d5; }

.chart-stats { display: flex; gap: 16px; font-size: 0.82rem; color:
var(--text-secondary); }
.chart-stats strong { color: var(--text-primary); font-family: Consolas,
'Courier New', monospace; }

.canvas-container {
    width: 100%;
    height: 220px;
    background: rgba(0, 0, 0, 0.16);
    border-radius: var(--radius-md);
    border: 1px solid rgba(255, 255, 255, 0.04);
    overflow: hidden;
}

canvas { width: 100%; height: 100%; display: block; }
```

```
/* Process Table */
.table-actions { display: flex; align-items: center; gap: 12px; flex-wrap:
wrap; }
.search-box {
    display: flex;
    align-items: center;
    gap: 8px;
    background: rgba(255, 255, 255, 0.05);
    border: 1px solid var(--border-color);
    border-radius: var(--radius-md);
    padding: 6px 12px;
    color: var(--text-secondary);
}
.search-box input {
    background: transparent; border: none; outline: none;
    color: var(--text-primary); font-family: inherit; font-size: 0.85rem;
width: 200px;
}

.form-select, .form-input-num {
    background: rgba(255, 255, 255, 0.05);
    border: 1px solid var(--border-color);
    border-radius: var(--radius-md);
    padding: 7px 12px;
    color: var(--text-primary);
    font-family: inherit;
    font-size: 0.85rem;
    outline: none;
}
.form-select option { background: #0f172a; color: var(--text-primary); }

.table-responsive {
    overflow-x: auto;
    max-height: 440px;
    border: 1px solid var(--border-color);
    border-radius: var(--radius-md);
    background: rgba(0, 0, 0, 0.2);
}
```

```
.data-table { width: 100%; border-collapse: collapse; font-size: 0.85rem;
text-align: left; }
.data-table th {
    background: rgba(14, 20, 34, 0.95);
    padding: 12px 16px;
    font-weight: 600;
    color: var(--text-secondary);
    border-bottom: 1px solid var(--border-color);
    position: sticky;
    top: 0;
    z-index: 10;
}
.data-table td { padding: 12px 16px; border-bottom: 1px solid rgba(255,
255, 255, 0.04); vertical-align: middle; }
.data-table tr:hover td { background: rgba(255, 255, 255, 0.03); }

.priority-tag {
    display: inline-block;
    padding: 3px 8px;
    border-radius: 4px;
    font-size: 0.72rem;
    font-weight: 700;
    font-family: Consolas, 'Courier New', monospace;
}
.priority-tag.REALTIME { background: rgba(244, 63, 94, 0.2); color:
#fda4af; border: 1px solid rgba(244, 63, 94, 0.4); }
.priority-tag.HIGH { background: rgba(245, 158, 11, 0.2); color: #fde68a;
border: 1px solid rgba(245, 158, 11, 0.4); }
.priority-tag.ABOVE_NORMAL { background: rgba(66, 133, 244, 0.2); color:
#93c5fd; border: 1px solid rgba(66, 133, 244, 0.4); }
.priority-tag.NORMAL { background: rgba(29, 185, 84, 0.2); color: #86efac;
border: 1px solid rgba(29, 185, 84, 0.4); }
.priority-tag.BELOW_NORMAL { background: rgba(14, 165, 233, 0.2); color:
#bae6fd; border: 1px solid rgba(14, 165, 233, 0.4); }
.priority-tag.IDLE { background: rgba(148, 163, 184, 0.2); color: #cbd5e1;
border: 1px solid rgba(148, 163, 184, 0.4); }

.memory-bar-cell { display: flex; align-items: center; gap: 10px; }
.memory-mini-bar { width: 50px; height: 5px; background: rgba(255, 255,
255, 0.08); border-radius: 2px; overflow: hidden; }
```

```
.memory-mini-fill { height: 100%; background: var(--purple-accent); }

/* Automotive Presets Bar */
.automotive-presets-bar {
    display: flex;
    align-items: center;
    gap: 10px;
    padding: 12px 16px;
    background: rgba(0, 0, 0, 0.35);
    border-radius: var(--radius-md);
    margin-bottom: 16px;
    flex-wrap: wrap;
    border: 1px solid var(--border-color);
}
.preset-label { font-size: 0.8rem; font-weight: 700; color:
var(--text-secondary); }

/* Scheduler Controls */
.scheduler-controls-bar {
    display: flex;
    justify-content: space-between;
    align-items: center;
    padding: 14px 16px;
    background: rgba(0, 0, 0, 0.16);
    border-radius: var(--radius-md);
    margin-bottom: 20px;
    flex-wrap: wrap;
    gap: 16px;
}
.control-group { display: flex; align-items: center; gap: 10px; font-size:
0.88rem; }
.checkbox-label { display: flex; align-items: center; gap: 8px; cursor:
pointer; font-size: 0.85rem; }
.control-actions { display: flex; align-items: center; gap: 12px;
flex-wrap: wrap; }
.sim-input-wrapper { max-height: 220px; margin-bottom: 24px; }
.input-cell {
    background: rgba(255, 255, 255, 0.06); border: 1px solid rgba(255,
255, 255, 0.1);
```

```
    color: var(--text-primary); padding: 4px 8px; border-radius: 4px;
width: 80px;
    font-family: Consolas, 'Courier New', monospace; font-size: 0.85rem;
}
.btn-remove-proc { background: none; border: none; color:
var(--rose-accent); cursor: pointer; font-size: 1.1rem; opacity: 0.7; }
.btn-remove-proc:hover { opacity: 1; }

/* Gantt Chart */
.gantt-section {
    margin-bottom: 24px;
    background: rgba(0, 0, 0, 0.3);
    padding: 20px;
    border-radius: var(--radius-md);
    border: 1px solid var(--border-color);
}
.gantt-header { display: flex; justify-content: space-between;
align-items: center; margin-bottom: 16px; }
.subsection-title { font-size: 0.95rem; font-weight: 600; }
.gantt-wrapper { position: relative; overflow-x: auto; padding-bottom:
10px; }
.gantt-chart {
    display: flex; height: 48px; border-radius: var(--radius-sm);
overflow: hidden;
    background: rgba(255, 255, 255, 0.04); border: 1px solid
var(--border-color); min-width: 600px;
}
.gantt-block {
    display: flex; flex-direction: column; align-items: center;
justify-content: center;
    height: 100%; color: #fff; font-size: 0.78rem; font-weight: 700;
    border-right: 1px solid rgba(0, 0, 0, 0.3); padding: 0 4px;
white-space: nowrap; overflow: hidden; text-overflow: ellipsis;
}
.gantt-block.idle { background: rgba(100, 116, 139, 0.2) !important;
color: #94a3b8; font-style: italic; }
.gantt-timeline-ticks { display: flex; position: relative; height: 24px;
min-width: 600px; margin-top: 4px; }
```

```
.gantt-tick { position: absolute; transform: translateX(-50%); font-size:
0.72rem; font-family: Consolas, 'Courier New', monospace; color:
var(--text-muted); }

/* Scoreboard */
.metrics-scoreboard {
    display: grid;
    grid-template-columns: repeat(auto-fit, minmax(200px, 1fr));
    gap: 16px;
    margin-bottom: 24px;
}
.score-card { background: rgba(255, 255, 255, 0.03); border: 1px solid
var(--border-color); border-radius: var(--radius-md); padding: 16px;
text-align: center; }
.score-label { font-size: 0.76rem; color: var(--text-secondary);
text-transform: uppercase; font-weight: 600; margin-bottom: 6px; }
.score-value { font-size: 1.8rem; font-weight: 700; font-family: Consolas,
'Courier New', monospace; color: var(--cyan-accent); }
.score-sub { font-size: 0.72rem; color: var(--text-muted); margin-top:
4px; }

/* Database Section */
.db-badges { display: flex; align-items: center; gap: 12px; }
.badge-db { background: var(--spotify-glow); color: var(--spotify-green);
border: 1px solid rgba(29, 185, 84, 0.4); padding: 4px 10px;
border-radius: 999px; font-size: 0.78rem; font-weight: 600; }
.db-details-grid { display: grid; grid-template-columns: 1fr 1.5fr; gap:
20px; }
@media (max-width: 900px) { .db-details-grid { grid-template-columns: 1fr;
} }
.db-info-panel { background: rgba(0, 0, 0, 0.16); border-radius:
var(--radius-md); padding: 20px; border: 1px solid var(--border-color); }
.info-panel-title { font-size: 0.92rem; margin-bottom: 14px; font-weight:
600; }
.db-table-list { list-style: none; display: flex; flex-direction: column;
gap: 12px; margin-bottom: 20px; }
.db-table-list li { font-size: 0.85rem; padding-left: 18px; position:
relative; }
.table-dot { position: absolute; left: 0; top: 6px; width: 6px; height:
6px; border-radius: 50%; background-color: var(--cyan-accent); }
```

```
.table-desc { font-size: 0.76rem; color: var(--text-muted); }
.db-actions-row { display: flex; gap: 10px; }
.db-sql-preview { background: #080c14; border-radius: var(--radius-md);
border: 1px solid var(--border-color); overflow: hidden; display: flex;
flex-direction: column; }
.code-header { background: rgba(255, 255, 255, 0.04); padding: 10px 16px;
border-bottom: 1px solid var(--border-color); display: flex;
justify-content: space-between; align-items: center; font-size: 0.78rem;
color: var(--text-secondary); }
.copy-badge { cursor: pointer; background: rgba(255, 255, 255, 0.08);
padding: 2px 8px; border-radius: 4px; font-size: 0.72rem; }
.copy-badge:hover { background: var(--cyan-accent); color: #000; }
.code-block { padding: 16px; overflow: auto; max-height: 240px; font-size:
0.78rem; color: #cbd5e1; line-height: 1.6; }

/* Modal */
.modal-overlay { position: fixed; inset: 0; background: rgba(0, 0, 0,
0.8); backdrop-filter: blur(8px); display: none; align-items: center;
justify-content: center; z-index: 999; }
.modal-overlay.active { display: flex; }
.modal-card { background: #0c1220; border: 1px solid var(--border-color);
border-radius: var(--radius-lg); width: 100%; max-width: 520px;
box-shadow: 0 20px 50px rgba(0, 0, 0, 0.7); overflow: hidden; }
.modal-header { padding: 20px 24px; border-bottom: 1px solid
var(--border-color); display: flex; justify-content: space-between;
align-items: center; }
.modal-title { font-size: 1.15rem; font-weight: 700; }
.modal-subtitle { font-size: 0.82rem; color: var(--text-secondary);
margin-top: 2px; }
.modal-close { background: none; border: none; font-size: 1.5rem; color:
var(--text-secondary); cursor: pointer; }
.modal-body { padding: 24px; }
.modal-desc { font-size: 0.85rem; color: var(--text-secondary);
margin-bottom: 16px; }
.priority-options { display: flex; flex-direction: column; gap: 10px; }
.priority-option { display: flex; align-items: center; gap: 12px; padding:
10px 14px; border: 1px solid var(--border-color); border-radius:
var(--radius-md); cursor: pointer; transition: var(--transition); }
.priority-option:hover { background: rgba(255, 255, 255, 0.04);
border-color: rgba(255, 255, 255, 0.15); }
```

```
.prio-info { display: flex; flex-direction: column; }
.prio-name { font-weight: 600; font-size: 0.88rem; }
.prio-desc { font-size: 0.74rem; color: var(--text-muted); }
.modal-feedback { margin-top: 14px; padding: 10px 14px; border-radius:
var(--radius-sm); font-size: 0.82rem; }
.modal-footer { padding: 16px 24px; border-top: 1px solid
var(--border-color); display: flex; justify-content: flex-end; gap: 12px;
background: rgba(0, 0, 0, 0.2); }

/* Alerts */
.alerts-container { display: flex; flex-direction: column; gap: 10px; }
.alert-banner { display: flex; align-items: center; justify-content:
space-between; padding: 12px 18px; border-radius: var(--radius-md);
font-size: 0.88rem; }
.alert-warning { background: rgba(245, 158, 11, 0.15); border: 1px solid
rgba(245, 158, 11, 0.4); color: #fde68a; }
.alert-danger { background: rgba(244, 63, 94, 0.15); border: 1px solid
rgba(244, 63, 94, 0.4); color: #fecdd3; }
.alert-success { background: rgba(16, 185, 129, 0.15); border: 1px solid
rgba(16, 185, 129, 0.4); color: #a7f3d0; }
.alert-close { cursor: pointer; background: none; border: none; color:
inherit; font-size: 1.1rem; opacity: 0.7; }

/* Utilities */
.text-emerald { color: var(--emerald-accent) !important; }
.footer { text-align: center; margin-top: 40px; font-size: 0.78rem; color:
var(--text-muted); }
```

SHOWING APPLICATION IN IT

```
//
================================================================================
===
// CarInfo - In-Vehicle Infotainment (IVI) Dashboard Client
// Subsystems: Google Maps Navigation | Spotify Media | Rear Camera |
CAN-Bus
//
================================================================================
===

(function () {
    'use strict';

    const API_BASE = window.location.origin.startsWith('http') ?
window.location.origin : 'http://localhost:8080';
    let isConnected = false;
    let peakCpu = 0;

    // Telemetry Rolling History (60 data points)
    const MAX_HISTORY = 60;
    const cpuHistory = new Array(MAX_HISTORY).fill(0);
    const ramHistory = new Array(MAX_HISTORY).fill(0);

    // Active Processes Cache
    let activeProcesses = [];
    let selectedProcForPriority = null;

    // Spotify Tracks
    const spotifyPlaylist = [
        { title: "Blinding Lights", artist: "The Weeknd • After Hours",
duration: "3:20" },
```

```
        { title: "Nightcall", artist: "Kavinsky • Drive Automotive OST",
duration: "4:19" },
        { title: "Starboy", artist: "The Weeknd ft. Daft Punk", duration:
"3:50" },
        { title: "Midnight City", artist: "M83 • Hurry Up, We're
Dreaming", duration: "4:03" }
    ];
    let currentTrackIdx = 0;
    let isPlaying = true;

    // Automotive Simulation Processes
    let simProcesses = [
        { pid: 1005, name: "rear_camera_feed", at: 0, bt: 4, prio: 1 },
        { pid: 1042, name: "google_maps_nav", at: 1, bt: 6, prio: 2 },
        { pid: 1088, name: "spotify_audio_dsp", at: 2, bt: 5, prio: 3 },
        { pid: 1010, name: "can_bus_gateway", at: 3, bt: 2, prio: 1 },
        { pid: 1060, name: "hvac_climate_ui", at: 4, bt: 3, prio: 4 }
    ];
    let currentSchedulerMode = 'RR';

    // DOM Elements - Vehicle Cockpit
    const clockDisplay = document.getElementById('clock-display');
    const backendStatusText =
document.getElementById('backend-status-text');
    const alertsContainer = document.getElementById('alerts-container');
    const hudSpeed = document.getElementById('hud-speed');
    const hudBattery = document.getElementById('hud-battery');
    const hudCabin = document.getElementById('hud-cabin');
    const gearSelector = document.getElementById('gear-selector');

    // KPI Elements
    const cpuPercentVal = document.getElementById('cpu-percent-val');
    const cpuBar = document.getElementById('cpu-bar');
    const cpuPeakBadge = document.getElementById('cpu-peak-badge');
    const cpuStateLabel = document.getElementById('cpu-state-label');

    const ramUsedVal = document.getElementById('ram-used-val');
    const ramTotalVal = document.getElementById('ram-total-val');
    const ramPercentBadge = document.getElementById('ram-percent-badge');
    const ramBar = document.getElementById('ram-bar');
```

```
    const ramFreeVal = document.getElementById('ram-free-val');

    const canBusVal = document.getElementById('can-bus-val');
    const canBar = document.getElementById('can-bar');
    const procCountVal = document.getElementById('proc-count-val');
    const uptimeVal = document.getElementById('uptime-val');
    const timestampVal = document.getElementById('timestamp-val');

    // Chart Elements
    const chartCpuCurrent = document.getElementById('chart-cpu-current');
    const chartCpuAvg = document.getElementById('chart-cpu-avg');
    const chartRamCurrent = document.getElementById('chart-ram-current');
    const chartRamUsedMb = document.getElementById('chart-ram-used-mb');
    const cpuCanvas = document.getElementById('cpu-chart');
    const ramCanvas = document.getElementById('ram-chart');

    // Google Maps Elements
    const btnRecalcRoute = document.getElementById('btn-recalc-route');

    // Spotify Elements
    const trackTitle = document.getElementById('track-title');
    const trackArtist = document.getElementById('track-artist');
    const btnPlayPause = document.getElementById('btn-play-pause');
    const btnNextTrack = document.getElementById('btn-next-track');
    const btnPrevTrack = document.getElementById('btn-prev-track');
    const audioWaveform = document.getElementById('audio-waveform');

    // Rear Camera Widget
    const rearCameraWidget =
document.getElementById('rear-camera-widget');
    const cameraStatusLabel =
document.getElementById('camera-status-label');
    const cameraMessage = document.getElementById('camera-message');
    const btnTriggerReverse =
document.getElementById('btn-trigger-reverse');

    // Process Table Elements
    const procTableBody = document.getElementById('process-table-body');
    const procSearchInput = document.getElementById('proc-search-input');
    const priorityFilter = document.getElementById('priority-filter');
```

```
    const btnRefreshProcs = document.getElementById('btn-refresh-procs');

    // Priority Modal Elements
    const priorityModal = document.getElementById('priority-modal');
    const modalProcInfo = document.getElementById('modal-proc-info');
    const modalFeedback = document.getElementById('modal-feedback');
    const btnCloseModal = document.getElementById('btn-close-modal');
    const btnCancelModal = document.getElementById('btn-cancel-modal');
    const btnConfirmPriority =
document.getElementById('btn-confirm-priority');

    // Scheduler Studio Elements
    const btnModeRr = document.getElementById('btn-mode-rr');
    const btnModePriority = document.getElementById('btn-mode-priority');
    const rrQuantumGroup = document.getElementById('rr-quantum-group');
    const priorityPreemptGroup =
document.getElementById('priority-preempt-group');
    const timeQuantumInput =
document.getElementById('time-quantum-input');
    const preemptiveCheckbox =
document.getElementById('preemptive-checkbox');
    const simInputTableBody =
document.getElementById('sim-input-table-body');
    const btnAddSimProc = document.getElementById('btn-add-sim-proc');
    const btnRunSimulation =
document.getElementById('btn-run-simulation');

    // Presets
    const presetReverse = document.getElementById('preset-reverse');
    const presetMultiapp = document.getElementById('preset-multiapp');
    const presetCan = document.getElementById('preset-can');

    const ganttAlgoBadge = document.getElementById('gantt-algo-badge');
    const ganttChart = document.getElementById('gantt-chart');
    const ganttTicks = document.getElementById('gantt-ticks');
    const scoreAvgTat = document.getElementById('score-avg-tat');
    const scoreAvgWt = document.getElementById('score-avg-wt');
    const scoreCpuUtil = document.getElementById('score-cpu-util');
    const scoreThroughput = document.getElementById('score-throughput');
```

```
    const simOutputTableBody =
document.getElementById('sim-output-table-body');

    // Database Sync Elements
    const btnSyncDbTop = document.getElementById('btn-sync-db-top');
    const btnSyncDbBottom = document.getElementById('btn-sync-db-bottom');
    const btnViewSql = document.getElementById('btn-view-sql');
    const btnDownloadSql = document.getElementById('btn-download-sql');
    const btnCopySql = document.getElementById('btn-copy-sql');
    const sqlPreviewCode = document.getElementById('sql-preview-code');
    const dbStatusBadge = document.getElementById('db-status-badge');

    // Colors for Automotive Gantt Timeline
    const GANTT_COLORS = [
        '#c45b64', '#4e83c4', '#4e9b78',
        '#c39245', '#9273bd', '#359eaa'
    ];

    //
================================================================================
    // Utilities & Alerts
    //
================================================================================

    function updateClock() {
        const now = new Date();
        clockDisplay.textContent = now.toTimeString().split(' ')[0];
    }
    setInterval(updateClock, 1000);
    updateClock();

    function showAlert(message, type = 'warning', timeout = 5000) {
        const alertDiv = document.createElement('div');
        alertDiv.className = `alert-banner alert-${type}`;
        alertDiv.innerHTML = `
            <span>${message}</span>
            <button class="alert-close">&times;</button>
        `;
        alertsContainer.appendChild(alertDiv);
```

```
        alertDiv.querySelector('.alert-close').addEventListener('click',
() => {
            alertDiv.remove();
        });

        if (timeout > 0) {
            setTimeout(() => {
                if (alertDiv.parentNode) alertDiv.remove();
            }, timeout);
        }
    }

    function formatUptime(seconds) {
        const h = Math.floor(seconds / 3600);
        const m = Math.floor((seconds % 3600) / 60);
        const s = Math.floor(seconds % 60);
        return `${h}h ${m < 10 ? '0' : ''}${m}m ${s < 10 ? '0' :
''}${s}s`;
    }

    //
================================================================================
    // Real-Time Canvas Rendering
    //
================================================================================

    function drawLineChart(canvas, data, color, glowColor, maxVal = 100) {
        if (!canvas) return;
        const ctx = canvas.getContext('2d');
        const width = canvas.width;
        const height = canvas.height;
        const padding = 12;

        ctx.clearRect(0, 0, width, height);

        // Draw a simple grid to make the values easier to compare.
        ctx.strokeStyle = 'rgba(255, 255, 255, 0.12)';
        ctx.lineWidth = 1;
        for (let i = 0; i <= 4; i++) {
            const y = padding + (height - padding * 2) * i / 4;
```

```
            ctx.beginPath();
            ctx.moveTo(0, y);
            ctx.lineTo(width, y);
            ctx.stroke();
        }

        if (data.length === 0) return;

        const step = data.length > 1 ? width / (data.length - 1) : 0;
        const pointY = (value) => height - padding - (Math.max(0,
Math.min(value, maxVal)) / maxVal) * (height - padding * 2);

        ctx.beginPath();
        data.forEach((value, i) => {
            const x = i * step;
            const y = pointY(value);
            if (i === 0) ctx.moveTo(x, y);
            else ctx.lineTo(x, y);
        });
        ctx.strokeStyle = color;
        ctx.lineWidth = 2;
        ctx.lineJoin = 'round';
        ctx.lineCap = 'round';
        ctx.stroke();

        // Mark the newest reading without adding a glow effect.
        const lastX = (data.length - 1) * step;
        const lastY = pointY(data[data.length - 1]);
        ctx.beginPath();
        ctx.arc(lastX, lastY, 3, 0, Math.PI * 2);
        ctx.fillStyle = color;
        ctx.fill();
    }

    //
================================================================================
    // Telemetry & Vehicle Polling
    //
================================================================================
```

```
    async function fetchVehicleTelemetry() {
        try {
            const res = await fetch(`${API_BASE}/api/vehicle/telemetry`);
            if (!res.ok) throw new Error("HTTP " + res.status);
            const data = await res.json();

            hudSpeed.textContent = data.speed_kmh;
            hudBattery.textContent = `${data.battery_percent}%`;
            hudCabin.textContent = `${data.cabin_temp_c}°C`;
            canBusVal.textContent = data.bus_load_percent.toFixed(1);
            canBar.style.width = `${Math.min(data.bus_load_percent,
100)}%`;

            // Update gear buttons
            gearSelector.querySelectorAll('.gear-btn').forEach(btn => {
                btn.classList.toggle('active',
btn.getAttribute('data-gear') === data.gear);
            });

            // Handle Reverse Gear Effects
            if (data.gear === 'R') {
                rearCameraWidget.classList.add('reverse-active');
                cameraStatusLabel.textContent = "ACTIVE FEED (REALTIME)";
                cameraStatusLabel.style.color = "#fda4af";
                cameraMessage.innerHTML = `<span style="color:#fecdd3;
font-weight:700;">⚠️ REVERSE ENGAGED: Safety Camera active with REALTIME
OS Priority</span>`;
            } else {
                rearCameraWidget.classList.remove('reverse-active');
                cameraStatusLabel.textContent = `Standby (Gear
${data.gear})`;
                cameraStatusLabel.style.color = "";
                cameraMessage.innerHTML = `<span>Gear in ${data.gear}:
Camera sleeping to preserve IVI CPU</span>
                <button class="btn btn-sm btn-outline"
id="btn-trigger-reverse-dyn">Simulate Shift to Reverse (R)</button>`;

                const btnDyn =
document.getElementById('btn-trigger-reverse-dyn');
                if (btnDyn) {
```

```
                btnDyn.addEventListener('click', () => setGear('R'));
            }
        }

    } catch (err) {
        // Local simulation fallback
        hudSpeed.textContent = "68";
        hudBattery.textContent = "85%";
        hudCabin.textContent = "21.5°C";
    }
}

async function fetchSystemMetrics() {
    try {
        const res = await fetch(`${API_BASE}/api/metrics`);
        if (!res.ok) throw new Error("HTTP " + res.status);
        const data = await res.json();

        isConnected = true;
        backendStatusText.textContent = "CAN-Bus Active";

        const cpu = Number(data.cpu_percent.toFixed(1));
        cpuPercentVal.textContent = cpu.toFixed(1);
        cpuBar.style.width = `${Math.min(cpu, 100)}%`;

        if (cpu > peakCpu) {
            peakCpu = cpu;
            cpuPeakBadge.textContent = `Peak: ${peakCpu.toFixed(1)}%`;
        }

        if (cpu > 80) {
            cpuStateLabel.textContent = "High Load";
            cpuStateLabel.className = "state-critical";
        } else if (cpu > 50) {
            cpuStateLabel.textContent = "Moderate";
            cpuStateLabel.className = "state-warning";
        } else {
            cpuStateLabel.textContent = "Normal";
            cpuStateLabel.className = "state-normal";
        }
```

```
            const ramUsed = Math.round(data.ram_used_mb);
            const ramTotal = Math.round(data.ram_total_mb);
            const ramPct = Number(data.ram_percent.toFixed(1));
            const ramFree = Math.max(0, ramTotal - ramUsed);

            ramUsedVal.textContent = ramUsed.toLocaleString();
            ramTotalVal.textContent = ramTotal.toLocaleString();
            ramPercentBadge.textContent = `${ramPct}%`;
            ramBar.style.width = `${Math.min(ramPct, 100)}%`;
            ramFreeVal.textContent = `${ramFree.toLocaleString()} MB
free`;

            procCountVal.textContent = data.process_count;
            uptimeVal.textContent = formatUptime(data.uptime_seconds);
            timestampVal.textContent = `Updated: ${data.timestamp.split('
')[1] || ''}`;

            cpuHistory.shift();
            cpuHistory.push(cpu);
            ramHistory.shift();
            ramHistory.push(ramPct);

            chartCpuCurrent.textContent = `${cpu.toFixed(1)}%`;
            const avgCpu = (cpuHistory.reduce((a, b) => a + b, 0) /
cpuHistory.length).toFixed(1);
            chartCpuAvg.textContent = `${avgCpu}%`;

            chartRamCurrent.textContent = `${ramPct}%`;
            chartRamUsedMb.textContent = `${ramUsed} MB`;

            drawLineChart(cpuCanvas, cpuHistory, '#08b6c9',
'transparent');
            drawLineChart(ramCanvas, ramHistory, '#9b83d5',
'transparent');

        } catch (err) {
            // Standalone Fallback
            const synthCpu = +(Math.random() * 15 + 18).toFixed(1);
            cpuPercentVal.textContent = synthCpu;
```

```
            cpuBar.style.width = `${synthCpu}%`;
            ramUsedVal.textContent = "6,850";
            ramTotalVal.textContent = "16,384";
            ramPercentBadge.textContent = "41.8%";
            ramBar.style.width = "41.8%";
            procCountVal.textContent = "7";
            uptimeVal.textContent = "1h 12m";

            cpuHistory.shift();
            cpuHistory.push(synthCpu);
            ramHistory.shift();
            ramHistory.push(41.8);

            drawLineChart(cpuCanvas, cpuHistory, '#08b6c9',
'transparent');
            drawLineChart(ramCanvas, ramHistory, '#9b83d5',
'transparent');
        }
    }

    async function setGear(gear) {
        try {
            await fetch(`${API_BASE}/api/vehicle/gear`, {
                method: 'POST',
                headers: { 'Content-Type': 'application/json' },
                body: JSON.stringify({ gear })
            });
            fetchVehicleTelemetry();
            if (gear === 'R') {
                showAlert("🚗 Shifted into REVERSE (R): Rear camera
preempted background tasks to REALTIME!", "danger");
            } else {
                showAlert(`🚗 Gear shifted to ${gear}.`, "success");
            }
        } catch (err) {
            // Local UI simulation
            gearSelector.querySelectorAll('.gear-btn').forEach(btn => {
                btn.classList.toggle('active',
btn.getAttribute('data-gear') === gear);
            });
```

```
            if (gear === 'R') {
                rearCameraWidget.classList.add('reverse-active');
                showAlert("🚗 Shifted into REVERSE (R): Rear camera
preempted background tasks to REALTIME!", "danger");
            } else {
                rearCameraWidget.classList.remove('reverse-active');
            }
        }
    }

    async function fetchProcessList() {
        try {
            const res = await fetch(`${API_BASE}/api/processes`);
            if (!res.ok) return;
            const data = await res.json();
            activeProcesses = data;
            renderProcessTable();
        } catch (err) {
            if (activeProcesses.length === 0) {
                activeProcesses = [
                    { pid: 1042, name: "google_maps_nav.bin", memory_mb:
438.5, thread_count: 24, priority_class: "ABOVE_NORMAL", subsystem:
"Navigation (Google Maps GPS)" },
                    { pid: 1088, name: "spotify_audio_dsp.bin", memory_mb:
194.2, thread_count: 16, priority_class: "NORMAL", subsystem: "Audio Media
(Spotify Stream)" },
                    { pid: 1005, name: "rear_camera_feed.bin", memory_mb:
88.0, thread_count: 8, priority_class: "REALTIME", subsystem: "Safety
(Rear-View HD Stream)" },
                    { pid: 1010, name: "can_bus_gateway.bin", memory_mb:
42.1, thread_count: 6, priority_class: "HIGH", subsystem: "CAN-Bus
Telemetry Gateway" },
                    { pid: 1060, name: "hvac_climate_ui.bin", memory_mb:
96.4, thread_count: 10, priority_class: "NORMAL", subsystem: "HVAC Cabin
Climate Control" },
                    { pid: 1018, name: "instrument_cluster_sync.bin",
memory_mb: 64.8, thread_count: 12, priority_class: "HIGH", subsystem:
"Virtual Cockpit & HUD Sync" },
```

```
                    { pid: 1120, name: "ota_telematics_sync.bin",
memory_mb: 112.0, thread_count: 8, priority_class: "IDLE", subsystem:
"Cloud Diagnostics & OTA Sync" }
                ];
                renderProcessTable();
            }
        }
    }

    //
================================================================================
    // Process Table & Priority Management
    //
================================================================================

    function renderProcessTable() {
        const query = (procSearchInput.value || '').toLowerCase().trim();
        const prioFilter = priorityFilter.value;

        const filtered = activeProcesses.filter(p => {
            const matchQuery = p.name.toLowerCase().includes(query) ||
                               (p.subsystem &&
p.subsystem.toLowerCase().includes(query)) ||
                               String(p.pid).includes(query);
            const matchPrio = (prioFilter === 'ALL' || p.priority_class
=== prioFilter);
            return matchQuery && matchPrio;
        });

        if (filtered.length === 0) {
            procTableBody.innerHTML = `<tr><td colspan="7"
class="loading-cell">No matching automotive processes found.</td></tr>`;
            return;
        }

        const maxMem = Math.max(...filtered.map(p => p.memory_mb), 1);

        procTableBody.innerHTML = filtered.map(p => {
            const memPercent = Math.min(100, Math.round((p.memory_mb /
maxMem) * 100));
```

```
            const category = p.subsystem || "Automotive Service";
            return `
                <tr>
                    <td class="mono"><strong>#${p.pid}</strong></td>
                    <td><strong>${escapeHtml(p.name)}</strong></td>
                    <td><span style="font-size:0.75rem;
color:#94a3b8;">${escapeHtml(category)}</span></td>
                    <td>
                        <div class="memory-bar-cell">
                            <div class="memory-mini-bar">
                                <div class="memory-mini-fill"
style="width: ${memPercent}%;"></div>
                            </div>
                            <span>${p.memory_mb.toFixed(1)} MB</span>
                        </div>
                    </td>
                    <td>${p.thread_count}</td>
                    <td><span class="priority-tag
${p.priority_class}">${p.priority_class}</span></td>
                    <td style="text-align: right;">
                        <button class="btn btn-outline btn-sm
btn-change-prio" data-pid="${p.pid}" data-name="${escapeHtml(p.name)}"
data-prio="${p.priority_class}">
                            Set Priority
                        </button>
                    </td>
                </tr>
            `;
        }).join('');

        procTableBody.querySelectorAll('.btn-change-prio').forEach(btn =>
{
            btn.addEventListener('click', () => {
                const pid = parseInt(btn.getAttribute('data-pid'), 10);
                const name = btn.getAttribute('data-name');
                const prio = btn.getAttribute('data-prio');
                openPriorityModal(pid, name, prio);
            });
        });
    }
```

```
    function openPriorityModal(pid, name, prio) {
        selectedProcForPriority = { pid, name, prio };
        modalProcInfo.textContent = `Subsystem: ${name} (PID: ${pid}) -
Current: ${prio}`;
        modalFeedback.style.display = 'none';

        const radios =
priorityModal.querySelectorAll('input[name="priority-choice"]');
        radios.forEach(r => {
            r.checked = (r.value === prio);
        });

        priorityModal.classList.add('active');
    }

    function closePriorityModal() {
        priorityModal.classList.remove('active');
        selectedProcForPriority = null;
    }

    async function applyProcessPriority() {
        if (!selectedProcForPriority) return;
        const selectedRadio =
priorityModal.querySelector('input[name="priority-choice"]:checked');
        if (!selectedRadio) return;

        const newPriority = selectedRadio.value;
        const pid = selectedProcForPriority.pid;

        btnConfirmPriority.disabled = true;
        btnConfirmPriority.textContent = "Applying...";

        try {
            const res = await fetch(`${API_BASE}/api/process/priority`, {
                method: 'POST',
                headers: { 'Content-Type': 'application/json' },
                body: JSON.stringify({ pid, priority: newPriority })
            });
```

```
            const data = await res.json();
            modalFeedback.style.display = 'block';

            if (data.success) {
                modalFeedback.className = 'modal-feedback alert-success';
                modalFeedback.textContent = data.message;
                showAlert(`✅ ${data.message}`, 'success');
                setTimeout(() => {
                    closePriorityModal();
                    fetchProcessList();
                }, 1200);
            } else {
                modalFeedback.className = 'modal-feedback alert-danger';
                modalFeedback.textContent = data.message;
            }
        } catch (err) {
            // Local simulation fallback
            modalFeedback.style.display = 'block';
            modalFeedback.className = 'modal-feedback alert-success';
            modalFeedback.textContent = `Simulation: Updated
${selectedProcForPriority.name} to ${newPriority}`;
            const target = activeProcesses.find(p => p.pid === pid);
            if (target) target.priority_class = newPriority;
            renderProcessTable();
            setTimeout(() => { closePriorityModal(); }, 1000);
        } finally {
            btnConfirmPriority.disabled = false;
            btnConfirmPriority.textContent = "Apply Priority Class";
        }
    }

    //
================================================================================
    // Spotify & Google Maps Interactive Controls
    //
================================================================================

    function updateSpotifyTrack() {
        const track = spotifyPlaylist[currentTrackIdx];
        trackTitle.textContent = track.title;
```

```
        trackArtist.textContent = track.artist;
        document.querySelector('.track-timestamps
span:last-child').textContent = track.duration;
    }

    function toggleSpotifyPlay() {
        isPlaying = !isPlaying;
        btnPlayPause.innerHTML = isPlaying ? '&#9208;' : '&#9654;';
        audioWaveform.querySelectorAll('.wave-bar').forEach(bar => {
            bar.style.animationPlayState = isPlaying ? 'running' :
'paused';
        });
    }

    function nextTrack() {
        currentTrackIdx = (currentTrackIdx + 1) % spotifyPlaylist.length;
        updateSpotifyTrack();
    }

    function prevTrack() {
        currentTrackIdx = (currentTrackIdx - 1 + spotifyPlaylist.length) %
spotifyPlaylist.length;
        updateSpotifyTrack();
    }

    function triggerRecalcRoute() {
        btnRecalcRoute.textContent = "Recalculating...";
        btnRecalcRoute.disabled = true;
        setTimeout(() => {
            btnRecalcRoute.textContent = "Recalculate Route";
            btnRecalcRoute.disabled = false;
            showAlert("🗺️ Google Maps: Found faster route via I-95 North
(Saves 3 mins). Priority ABOVE_NORMAL.", "success");
        }, 800);
    }

    //
============================================================================
    // Automotive CPU Scheduling Algorithm Studio
```

```
    //
================================================================================

    function renderSimInputTable() {
        simInputTableBody.innerHTML = simProcesses.map((p, idx) => `
            <tr>
                <td class="mono">#${p.pid}</td>
                <td><input type="text" class="input-cell" style="width:
150px;" value="${p.name}" data-idx="${idx}" data-field="name"></td>
                <td><input type="number" class="input-cell" min="0"
max="100" value="${p.at}" data-idx="${idx}" data-field="at"></td>
                <td><input type="number" class="input-cell" min="1"
max="50" value="${p.bt}" data-idx="${idx}" data-field="bt"></td>
                <td><input type="number" class="input-cell" min="1"
max="10" value="${p.prio}" data-idx="${idx}" data-field="prio"></td>
                <td style="text-align: center;">
                    <button class="btn-remove-proc"
data-idx="${idx}">&times;</button>
                </td>
            </tr>
        `).join('');

        simInputTableBody.querySelectorAll('.input-cell').forEach(inp => {
            inp.addEventListener('change', (e) => {
                const idx = parseInt(e.target.getAttribute('data-idx'),
10);
                const field = e.target.getAttribute('data-field');
                if (field === 'name') simProcesses[idx].name =
e.target.value;
                else simProcesses[idx][field] = parseInt(e.target.value,
10) || 0;
            });
        });

        simInputTableBody.querySelectorAll('.btn-remove-proc').forEach(btn
=> {
            btn.addEventListener('click', (e) => {
                const idx = parseInt(e.target.getAttribute('data-idx'),
10);
                simProcesses.splice(idx, 1);
```

```
                renderSimInputTable();
            });
        });
    }

    function loadScenarioReverse() {
        currentSchedulerMode = 'PRIORITY';
        btnModePriority.classList.add('active');
        btnModeRr.classList.remove('active');
        rrQuantumGroup.style.display = 'none';
        priorityPreemptGroup.style.display = 'flex';
        preemptiveCheckbox.checked = true;

        simProcesses = [
            { pid: 1042, name: "google_maps_nav", at: 0, bt: 6, prio: 3 },
            { pid: 1088, name: "spotify_audio_dsp", at: 1, bt: 5, prio: 4
},
            { pid: 1005, name: "rear_camera_feed", at: 2, bt: 4, prio: 1
}, // Shifts into R at t=2!
            { pid: 1010, name: "can_bus_gateway", at: 3, bt: 2, prio: 2 }
        ];

        renderSimInputTable();
        runSimulation();
        showAlert("🚗 Loaded Scenario 1: Reverse Gear Preemption. Notice
how rear_camera_feed (Prio 1) interrupts Google Maps and Spotify at t=2!",
"danger", 6000);
    }

    function loadScenarioMultiApp() {
        currentSchedulerMode = 'RR';
        btnModeRr.classList.add('active');
        btnModePriority.classList.remove('active');
        rrQuantumGroup.style.display = 'flex';
        priorityPreemptGroup.style.display = 'none';
        timeQuantumInput.value = 3;

        simProcesses = [
            { pid: 1042, name: "google_maps_nav", at: 0, bt: 7, prio: 2 },
```

```
            { pid: 1088, name: "spotify_audio_dsp", at: 1, bt: 5, prio: 3 },
            { pid: 1060, name: "hvac_climate_ui", at: 2, bt: 3, prio: 3 },
            { pid: 1018, name: "instrument_cluster", at: 3, bt: 4, prio: 2 }
        ];

        renderSimInputTable();
        runSimulation();
        showAlert("🎵 Loaded Scenario 2: Multi-App Round Robin. Apps share processor slices smoothly without touchscreen lag or audio stutter.", "success", 5000);
    }

    function loadScenarioCan() {
        currentSchedulerMode = 'PRIORITY';
        btnModePriority.classList.add('active');
        btnModeRr.classList.remove('active');
        rrQuantumGroup.style.display = 'none';
        priorityPreemptGroup.style.display = 'flex';
        preemptiveCheckbox.checked = true;

        simProcesses = [
            { pid: 1088, name: "spotify_audio_dsp", at: 0, bt: 6, prio: 4 },
            { pid: 1042, name: "google_maps_nav", at: 1, bt: 6, prio: 3 },
            { pid: 1010, name: "can_emergency_abs", at: 2, bt: 3, prio: 1 } // Emergency ABS trigger!
        ];

        renderSimInputTable();
        runSimulation();
        showAlert("⚠️ Loaded Scenario 3: CAN Emergency Interrupt. Critical vehicle safety packet interrupts Spotify instantly.", "warning", 5000);
    }

    function addSimProcess() {
        const nextId = 1000 + simProcesses.length + 1;
        simProcesses.push({
            pid: nextId,
```

```
            name: `ivi_service_${String.fromCharCode(65 +
simProcesses.length)}`,
            at: simProcesses.length,
            bt: Math.floor(Math.random() * 5) + 3,
            prio: Math.floor(Math.random() * 3) + 2
        });
        renderSimInputTable();
    }

    async function runSimulation() {
        const quantum = parseInt(timeQuantumInput.value, 10) || 3;
        const preemptive = preemptiveCheckbox.checked;

        const payload = {
            algorithm: currentSchedulerMode === 'RR' ? 'round_robin' :
'priority',
            quantum: quantum,
            preemptive: preemptive,
            processes: simProcesses
        };

        try {
            const res = await fetch(`${API_BASE}/api/schedule`, {
                method: 'POST',
                headers: { 'Content-Type': 'application/json' },
                body: JSON.stringify(payload)
            });

            if (!res.ok) throw new Error("HTTP " + res.status);
            const data = await res.json();
            renderSimulationResults(data);
        } catch (err) {
            // Client-side simulation fallback
            const localResult = currentSchedulerMode === 'RR'
                ? simulateRoundRobinClient(simProcesses, quantum)
                : simulatePriorityClient(simProcesses, preemptive);
            renderSimulationResults(localResult);
        }
    }
```

```
    function renderSimulationResults(data) {
        ganttAlgoBadge.textContent = data.algorithm === 'ROUND_ROBIN'
            ? `Round Robin (Quantum q=${data.time_quantum})`
            : (data.algorithm === 'PRIORITY_PREEMPTIVE' ? 'Priority
(Preemptive Safety Mode)' : 'Priority (Non-Preemptive)');

        const timeline = data.timeline || [];
        const totalDuration = timeline.length > 0 ?
timeline[timeline.length - 1].end_time : 1;

        ganttChart.innerHTML = '';
        ganttTicks.innerHTML = '';

        timeline.forEach((slice, idx) => {
            const duration = slice.end_time - slice.start_time;
            const widthPct = (duration / totalDuration) * 100;
            const block = document.createElement('div');
            block.className = `gantt-block ${slice.is_idle ? 'idle' :
''}`;
            block.style.width = `${widthPct}%`;

            // Specific colors for recognized IVI apps
            let blockBg = GANTT_COLORS[slice.pid % GANTT_COLORS.length];
            if (slice.name.includes('camera') ||
slice.name.includes('emergency')) blockBg = '#c45b64';
            else if (slice.name.includes('maps')) blockBg = '#4e83c4';
            else if (slice.name.includes('spotify')) blockBg = '#4e9b78';

            block.style.background = slice.is_idle ? 'transparent' :
blockBg;
            block.textContent = `${slice.name}
[${slice.start_time}-${slice.end_time}]`;
            block.title = `${slice.name}: Start t=${slice.start_time}, End
t=${slice.end_time} (Slice: ${duration})`;
            ganttChart.appendChild(block);

            if (idx === 0) {
                const tickStart = document.createElement('span');
                tickStart.className = 'gantt-tick';
                tickStart.style.left = '0%';
```

```
                tickStart.textContent = '0';
                ganttTicks.appendChild(tickStart);
            }

            const tickEnd = document.createElement('span');
            tickEnd.className = 'gantt-tick';
            const endPct = (slice.end_time / totalDuration) * 100;
            tickEnd.style.left = `${endPct}%`;
            tickEnd.textContent = slice.end_time;
            ganttTicks.appendChild(tickEnd);
        });

        scoreAvgTat.textContent = (data.avg_turnaround_time ||
0).toFixed(2);
        scoreAvgWt.textContent = (data.avg_waiting_time || 0).toFixed(2);
        scoreCpuUtil.textContent = `${(data.cpu_utilization ||
100).toFixed(1)}%`;
        scoreThroughput.textContent = (data.throughput || 0).toFixed(3);

        const procs = data.processes || [];
        simOutputTableBody.innerHTML = procs.map(p => `
            <tr>
                <td class="mono"><strong>#${p.pid}</strong></td>
                <td><strong>${escapeHtml(p.name)}</strong></td>
                <td>${p.arrival_time}</td>
                <td>${p.burst_time}</td>
                <td><span class="badge-accent">${p.priority}</span></td>
                <td class="mono">${p.completion_time}</td>
                <td class="mono font-bold">${p.turnaround_time}</td>
                <td class="mono">${p.waiting_time}</td>
                <td class="mono">${p.response_time}</td>
            </tr>
        `).join('');
    }

    // Client-side simulation fallbacks
    function simulateRoundRobinClient(procs, quantum) {
        const n = procs.length;
        const pList = procs.map(p => ({ ...p, rem: p.bt, ct: 0, tat: 0,
wt: 0, rt: -1 }));
```

```
        let cur = 0, done = 0;
        const queue = [], inQ = new Array(n).fill(false), timeline = [];

        pList.forEach((p, i) => { if (p.at <= cur) { queue.push(i); inQ[i]
= true; } });

        while (done < n) {
            if (queue.length === 0) {
                let nextAt = 99999;
                pList.forEach(p => { if (p.rem > 0 && p.at < nextAt)
nextAt = p.at; });
                if (nextAt > cur) {
                    timeline.push({ pid: 0, name: "IDLE", start_time: cur,
end_time: nextAt, is_idle: true });
                    cur = nextAt;
                }
                pList.forEach((p, i) => { if (p.rem > 0 && p.at <= cur &&
!inQ[i]) { queue.push(i); inQ[i] = true; } });
                continue;
            }

            const idx = queue.shift();
            inQ[idx] = false;
            if (pList[idx].rt === -1) pList[idx].rt = cur - pList[idx].at;

            const exec = Math.min(quantum, pList[idx].rem);
            timeline.push({ pid: pList[idx].pid, name: pList[idx].name,
start_time: cur, end_time: cur + exec, is_idle: false });
            cur += exec;
            pList[idx].rem -= exec;

            pList.forEach((p, i) => { if (p.rem > 0 && p.at <= cur &&
!inQ[i] && i !== idx) { queue.push(i); inQ[i] = true; } });

            if (pList[idx].rem > 0) {
                queue.push(idx);
                inQ[idx] = true;
            } else {
                done++;
                pList[idx].ct = cur;
```

```
                pList[idx].tat = pList[idx].ct - pList[idx].at;
                pList[idx].wt = pList[idx].tat - pList[idx].bt;
            }
        }

        const avgTat = pList.reduce((a, b) => a + b.tat, 0) / n;
        const avgWt = pList.reduce((a, b) => a + b.wt, 0) / n;

        return {
            algorithm: 'ROUND_ROBIN',
            time_quantum: quantum,
            avg_turnaround_time: avgTat,
            avg_waiting_time: avgWt,
            cpu_utilization: 100.0,
            throughput: n / cur,
            processes: pList.map(p => ({
                pid: p.pid, name: p.name, arrival_time: p.at, burst_time:
p.bt, priority: p.prio,
                completion_time: p.ct, turnaround_time: p.tat,
waiting_time: p.wt, response_time: p.rt
            })),
            timeline: timeline
        };
    }

    function simulatePriorityClient(procs, preemptive) {
        const n = procs.length;
        const pList = procs.map(p => ({ ...p, rem: p.bt, ct: 0, tat: 0,
wt: 0, rt: -1 }));
        let cur = 0, done = 0;
        const timeline = [];

        if (!preemptive) {
            const isDone = new Array(n).fill(false);
            while (done < n) {
                let bestIdx = -1, bestPrio = 99999;
                pList.forEach((p, i) => {
                    if (!isDone[i] && p.at <= cur) {
                        if (p.prio < bestPrio || (p.prio === bestPrio &&
p.at < pList[bestIdx].at)) {
```

```
                        bestPrio = p.prio;
                        bestIdx = i;
                    }
                }
            });

            if (bestIdx === -1) {
                let nextAt = 99999;
                pList.forEach((p, i) => { if (!isDone[i] && p.at <
nextAt) nextAt = p.at; });
                timeline.push({ pid: 0, name: "IDLE", start_time: cur,
end_time: nextAt, is_idle: true });
                cur = nextAt;
                continue;
            }

            const p = pList[bestIdx];
            p.rt = cur - p.at;
            timeline.push({ pid: p.pid, name: p.name, start_time: cur,
end_time: cur + p.bt, is_idle: false });
            cur += p.bt;
            isDone[bestIdx] = true;
            done++;
            p.ct = cur;
            p.tat = p.ct - p.at;
            p.wt = p.tat - p.bt;
        }
    } else {
        while (done < n) {
            let bestIdx = -1, bestPrio = 99999;
            pList.forEach((p, i) => {
                if (p.rem > 0 && p.at <= cur) {
                    if (p.prio < bestPrio || (p.prio === bestPrio &&
p.at < pList[bestIdx].at)) {
                        bestPrio = p.prio;
                        bestIdx = i;
                    }
                }
            });
```

```
                if (bestIdx === -1) {
                    timeline.push({ pid: 0, name: "IDLE", start_time: cur,
end_time: cur + 1, is_idle: true });
                    cur++;
                    continue;
                }

                const p = pList[bestIdx];
                if (p.rt === -1) p.rt = cur - p.at;
                timeline.push({ pid: p.pid, name: p.name, start_time: cur,
end_time: cur + 1, is_idle: false });
                p.rem--;
                cur++;
                if (p.rem === 0) {
                    done++;
                    p.ct = cur;
                    p.tat = p.ct - p.at;
                    p.wt = p.tat - p.bt;
                }
            }
        }

        const merged = [];
        timeline.forEach(sl => {
            if (merged.length > 0 && merged[merged.length - 1].pid ===
sl.pid && merged[merged.length - 1].is_idle === sl.is_idle) {
                merged[merged.length - 1].end_time = sl.end_time;
            } else {
                merged.push({ ...sl });
            }
        });

        const avgTat = pList.reduce((a, b) => a + b.tat, 0) / n;
        const avgWt = pList.reduce((a, b) => a + b.wt, 0) / n;

        return {
            algorithm: preemptive ? 'PRIORITY_PREEMPTIVE' :
'PRIORITY_NON_PREEMPTIVE',
            time_quantum: 0,
            avg_turnaround_time: avgTat,
```

```
            avg_waiting_time: avgWt,
            cpu_utilization: 100.0,
            throughput: n / cur,
            processes: pList.map(p => ({
                pid: p.pid, name: p.name, arrival_time: p.at, burst_time:
p.bt, priority: p.prio,
                completion_time: p.ct, turnaround_time: p.tat,
waiting_time: p.wt, response_time: p.rt
            })),
            timeline: merged
        };
    }

    //
================================================================================
    // PostgreSQL Sync
    //
================================================================================

    async function triggerDatabaseSync() {
        dbStatusBadge.textContent = "Syncing...";
        try {
            const res = await fetch(`${API_BASE}/api/db/sync`, { method:
'POST' });
            const data = await res.json();
            showAlert(`✅ PostgreSQL Sync: ${data.message}`, 'success');
            dbStatusBadge.textContent = "Synced (" +
data.records_generated + " records)";
            fetchLatestSqlDump();
        } catch (err) {
            showAlert("Automotive telemetry snapshot exported to
database/latest_snapshot.sql", "success");
            dbStatusBadge.textContent = "SQL Exported";
        }
    }

    async function fetchLatestSqlDump() {
        try {
            const res = await fetch(`${API_BASE}/api/db/dump`);
            if (res.ok) {
```

```
                const sql = await res.text();
                if (sql) sqlPreviewCode.textContent = sql;
            }
        } catch (err) {
            console.warn("Could not fetch SQL dump:", err);
        }
    }

    function escapeHtml(str) {
        if (!str) return '';
        return str.replace(/&/g, '&amp;').replace(/</g,
'&lt;').replace(/>/g, '&gt;').replace(/"/g, '&quot;');
    }

    //
================================================================================
    // Event Listeners & Initialization
    //
================================================================================

    function setupEventListeners() {
        // Gear Selector
        gearSelector.querySelectorAll('.gear-btn').forEach(btn => {
            btn.addEventListener('click', (e) => {
                const gear = e.target.getAttribute('data-gear');
                setGear(gear);
            });
        });

        if (btnTriggerReverse) {
            btnTriggerReverse.addEventListener('click', () =>
setGear('R'));
        }

        // Spotify
        btnPlayPause.addEventListener('click', toggleSpotifyPlay);
        btnNextTrack.addEventListener('click', nextTrack);
        btnPrevTrack.addEventListener('click', prevTrack);

        // Google Maps
```

```
        btnRecalcRoute.addEventListener('click', triggerRecalcRoute);

        // Process Table
        procSearchInput.addEventListener('input', renderProcessTable);
        priorityFilter.addEventListener('change', renderProcessTable);
        btnRefreshProcs.addEventListener('click', fetchProcessList);

        // Priority Modal
        btnCloseModal.addEventListener('click', closePriorityModal);
        btnCancelModal.addEventListener('click', closePriorityModal);
        btnConfirmPriority.addEventListener('click',
applyProcessPriority);

        // Scheduler Tabs
        btnModeRr.addEventListener('click', () => {
            currentSchedulerMode = 'RR';
            btnModeRr.classList.add('active');
            btnModePriority.classList.remove('active');
            rrQuantumGroup.style.display = 'flex';
            priorityPreemptGroup.style.display = 'none';
            runSimulation();
        });

        btnModePriority.addEventListener('click', () => {
            currentSchedulerMode = 'PRIORITY';
            btnModePriority.classList.add('active');
            btnModeRr.classList.remove('active');
            rrQuantumGroup.style.display = 'none';
            priorityPreemptGroup.style.display = 'flex';
            runSimulation();
        });

        btnAddSimProc.addEventListener('click', addSimProcess);
        btnRunSimulation.addEventListener('click', runSimulation);

        // Automotive Presets
        presetReverse.addEventListener('click', loadScenarioReverse);
        presetMultiapp.addEventListener('click', loadScenarioMultiApp);
        presetCan.addEventListener('click', loadScenarioCan);
```

```
        // DB Sync
        btnSyncDbTop.addEventListener('click', triggerDatabaseSync);
        btnSyncDbBottom.addEventListener('click', triggerDatabaseSync);
        btnViewSql.addEventListener('click', fetchLatestSqlDump);
        btnCopySql.addEventListener('click', () => {
            navigator.clipboard.writeText(sqlPreviewCode.textContent);
            btnCopySql.textContent = "Copied!";
            setTimeout(() => { btnCopySql.textContent = "Copy"; }, 2000);
        });

        btnDownloadSql.addEventListener('click', () => {
            const blob = new Blob([sqlPreviewCode.textContent], { type:
'text/sql' });
            const url = URL.createObjectURL(blob);
            const a = document.createElement('a');
            a.href = url;
            a.download = `carinfo_telemetry_${new
Date().toISOString().replace(/[:.]/g, '-')}.sql`;
            a.click();
            URL.revokeObjectURL(url);
        });
    }

    function init() {
        setupEventListeners();
        renderSimInputTable();
        runSimulation();
        updateSpotifyTrack();

        fetchVehicleTelemetry();
        fetchSystemMetrics();
        fetchProcessList();
        fetchLatestSqlDump();

        setInterval(fetchVehicleTelemetry, 1500);
        setInterval(fetchSystemMetrics, 1000);
        setInterval(fetchProcessList, 2500);
    }

    window.addEventListener('DOMContentLoaded', init);
```

```
})();
```

```
#include "http_server.h"
#include <iostream>
```

```
#include <fstream>
#include <sstream>
#include <regex>
#include <iomanip>

struct ClientSession {
    HttpServer* server;
    SOCKET socket;
};

static DWORD WINAPI clientSessionWorker(LPVOID lpParam) {
    ClientSession* session = static_cast<ClientSession*>(lpParam);
    if (session && session->server) {
        session->server->clientHandler(session->socket);
    }
    delete session;
    return 0;
}

HttpServer::HttpServer(int port,
                       SystemMonitor& monitor,
                       PriorityController& prioController,
                       CpuScheduler& scheduler,
                       DatabaseManager& dbManager)
    : port(port), serverSocket(INVALID_SOCKET), isRunning(false),
      monitor(monitor), prioController(prioController),
scheduler(scheduler), dbManager(dbManager) {}

HttpServer::~HttpServer() {
    stop();
}

static std::string extractStringField(const std::string& json, const
std::string& key) {
    std::string pattern = "\"" + key + "\"\\s*:\\s*\"([^\"]*)\"";
    std::regex re(pattern);
    std::smatch match;
    if (std::regex_search(json, match, re) && match.size() > 1) {
        return match[1].str();
    }
```

```
    return "";
}

static int extractIntField(const std::string& json, const std::string&
key, int defaultVal = 0) {
    std::string pattern = "\"" + key + "\"\\s*:\\s*(-?[0-9]+)";
    std::regex re(pattern);
    std::smatch match;
    if (std::regex_search(json, match, re) && match.size() > 1) {
        return std::stoi(match[1].str());
    }
    return defaultVal;
}

static bool extractBoolField(const std::string& json, const std::string&
key, bool defaultVal = false) {
    std::string pattern = "\"" + key + "\"\\s*:\\s*(true|false)";
    std::regex re(pattern);
    std::smatch match;
    if (std::regex_search(json, match, re) && match.size() > 1) {
        return match[1].str() == "true";
    }
    return defaultVal;
}

std::string HttpServer::readFileContent(const std::string& path) {
    std::ifstream file(path, std::ios::binary);
    if (!file.is_open()) return "";
    std::ostringstream ss;
    ss << file.rdbuf();
    return ss.str();
}

void HttpServer::sendResponse(SOCKET clientSocket, int statusCode, const
std::string& contentType, const std::string& body) {
    std::string statusText = (statusCode == 200) ? "OK" : (statusCode ==
404 ? "Not Found" : "Internal Server Error");
    std::stringstream header;
    header << "HTTP/1.1 " << statusCode << " " << statusText << "\r\n"
           << "Content-Type: " << contentType << "\r\n"
```

```
           << "Content-Length: " << body.size() << "\r\n"
           << "Access-Control-Allow-Origin: *\r\n"
           << "Access-Control-Allow-Methods: GET, POST, OPTIONS\r\n"
           << "Access-Control-Allow-Headers: Content-Type\r\n"
           << "Connection: close\r\n\r\n";

    std::string full = header.str() + body;
    send(clientSocket, full.c_str(), static_cast<int>(full.size()), 0);
}

std::string HttpServer::handleGetMetrics() {
    SystemMetrics m = monitor.getSystemMetrics();
    std::stringstream ss;
    ss << std::fixed << std::setprecision(2);
    ss << "{"
       << "\"cpu_percent\":" << m.cpu_percent << ","
       << "\"ram_used_mb\":" << m.ram_used_mb << ","
       << "\"ram_total_mb\":" << m.ram_total_mb << ","
       << "\"ram_percent\":" << m.ram_percent << ","
       << "\"process_count\":" << m.process_count << ","
       << "\"uptime_seconds\":" << m.uptime_seconds << ","
       << "\"timestamp\":\"" << m.timestamp << "\""
       << "}";
    return ss.str();
}

std::string HttpServer::handleGetVehicleTelemetry() {
    VehicleTelemetry v = monitor.getVehicleTelemetry();
    std::stringstream ss;
    ss << std::fixed << std::setprecision(1);
    ss << "{"
       << "\"speed_kmh\":" << v.speed_kmh << ","
       << "\"rpm\":" << v.rpm << ","
       << "\"battery_percent\":" << v.battery_percent << ","
       << "\"cabin_temp_c\":" << v.cabin_temp_c << ","
       << "\"gear\":\"" << v.gear << "\","
       << "\"reverse_camera_active\":" << (v.reverse_camera_active ?
"true" : "false") << ","
       << "\"driving_mode\":\"" << v.driving_mode << "\","
       << "\"bus_load_percent\":" << v.bus_load_percent << ","
```

```
       << "\"coolant_temp_c\":" << v.coolant_temp_c
       << "}";
    return ss.str();
}

std::string HttpServer::handlePostVehicleGear(const std::string& body) {
    std::string gear = extractStringField(body, "gear");
    if (gear.empty()) gear = "D";
    monitor.setVehicleGear(gear);
    return handleGetVehicleTelemetry();
}

std::string HttpServer::handleGetIviApps() {
    std::vector<ProcessInfo> procs = monitor.getIviProcessList();
    std::stringstream ss;
    ss << std::fixed << std::setprecision(2);
    ss << "[";
    for (size_t i = 0; i < procs.size(); ++i) {
        const auto& p = procs[i];
        if (i > 0) ss << ",";
        ss << "{"
           << "\"pid\":" << p.pid << ","
           << "\"name\":\"" << p.name << "\","
           << "\"memory_mb\":" << p.memory_mb << ","
           << "\"thread_count\":" << p.thread_count << ","
           << "\"priority_class\":\"" << p.priority_class << "\","
           << "\"subsystem\":\"" << p.subsystem << "\""
           << "}";
    }
    ss << "]";
    return ss.str();
}

std::string HttpServer::handleGetProcesses() {
    std::vector<ProcessInfo> procs = monitor.getProcessList(80);
    std::stringstream ss;
    ss << std::fixed << std::setprecision(2);
    ss << "[";
    for (size_t i = 0; i < procs.size(); ++i) {
        const auto& p = procs[i];
```

```
        if (i > 0) ss << ",";
        ss << "{"
           << "\"pid\":" << p.pid << ","
           << "\"name\":\"" << p.name << "\","
           << "\"memory_mb\":" << p.memory_mb << ","
           << "\"thread_count\":" << p.thread_count << ","
           << "\"priority_class\":\"" << p.priority_class << "\","
           << "\"subsystem\":\"" << p.subsystem << "\""
           << "}";
    }
    ss << "]";
    return ss.str();
}

std::string HttpServer::handlePostPriority(const std::string& body) {
    int pid = extractIntField(body, "pid", 0);
    std::string priority = extractStringField(body, "priority");
    if (priority.empty()) {
        priority = "NORMAL";
    }

    std::string message;
    bool ok = prioController.setProcessPriority(pid, priority, message);

    std::stringstream ss;
    ss << "{"
       << "\"success\":" << (ok ? "true" : "false") << ","
       << "\"pid\":" << pid << ","
       << "\"priority\":\"" << priority << "\","
       << "\"message\":\"" << message << "\","
       << "\"audit_logs\":" << prioController.getAuditLogsJson()
       << "}";
    return ss.str();
}

std::string HttpServer::handlePostSchedule(const std::string& body) {
    std::string algo = extractStringField(body, "algorithm");
    int quantum = extractIntField(body, "quantum", 2);
    bool preemptive = extractBoolField(body, "preemptive", false);
```

```
    std::vector<ScheduledProcess> list;

    // Parse processes array from body: {"processes":
[{"pid":1,"name":"P1","at":0,"bt":5,"prio":2}, ...]}
    std::regex
procObjRe("\\{\\s*\"pid\"\\s*:\\s*([0-9]+)\\s*,\\s*\"name\"\\s*:\\s*\"([^\
"]*)\"\\s*,\\s*\"at\"\\s*:\\s*([0-9]+)\\s*,\\s*\"bt\"\\s*:\\s*([0-9]+)\\s*
,\\s*\"prio\"\\s*:\\s*([0-9]+)\\s*\\}");
    std::sregex_iterator next(body.begin(), body.end(), procObjRe);
    std::sregex_iterator end;
    while (next != end) {
        std::smatch match = *next;
        ScheduledProcess p;
        p.pid = std::stoul(match[1].str());
        p.name = match[2].str();
        p.arrival_time = std::stoi(match[3].str());
        p.burst_time = std::stoi(match[4].str());
        p.priority = std::stoi(match[5].str());
        list.push_back(p);
        next++;
    }

    // Default automotive processes if none provided
    if (list.empty()) {
        list = {
            {1005, "rear_camera_feed", 0, 4, 1},
            {1042, "google_maps_nav", 1, 6, 2},
            {1088, "spotify_audio_dsp", 2, 5, 3},
            {1010, "can_bus_gateway", 3, 2, 1},
            {1060, "hvac_climate_ui", 4, 3, 4}
        };
    }

    SchedulingResult result;
    if (algo == "round_robin" || algo == "ROUND_ROBIN") {
        result = scheduler.runRoundRobin(list, quantum);
    } else {
        result = scheduler.runPriority(list, preemptive);
    }
```

```
    lastSimulations.push_back(result);
    return result.toJson();
}

std::string HttpServer::handlePostDbSync() {
    SystemMetrics m = monitor.getSystemMetrics();
    std::vector<ProcessInfo> procs = monitor.getIviProcessList();
    auto auditLogs = prioController.getAuditLogs();

    DbSyncResult res = dbManager.syncSnapshotToDatabase(m, procs,
auditLogs, lastSimulations);

    std::stringstream ss;
    ss << "{"
       << "\"success\":" << (res.success ? "true" : "false") << ","
       << "\"records_generated\":" << res.records_generated << ","
       << "\"message\":\"" << res.message << "\""
       << "}";
    return ss.str();
}

void HttpServer::clientHandler(SOCKET clientSocket) {
    char buffer[8192];
    int bytesRead = recv(clientSocket, buffer, sizeof(buffer) - 1, 0);
    if (bytesRead <= 0) {
        closesocket(clientSocket);
        return;
    }
    buffer[bytesRead] = '\0';
    std::string req(buffer);

    std::istringstream reqStream(req);
    std::string method, url, proto;
    reqStream >> method >> url >> proto;

    // Handle OPTIONS for CORS
    if (method == "OPTIONS") {
        sendResponse(clientSocket, 200, "text/plain", "OK");
        closesocket(clientSocket);
        return;
```

```
    }

    // Separate body from headers
    std::string body = "";
    size_t bodyPos = req.find("\r\n\r\n");
    if (bodyPos != std::string::npos) {
        body = req.substr(bodyPos + 4);
    }

    // Routing
    if (method == "GET") {
        if (url == "/" || url == "/index.html") {
            std::string content = readFileContent("public\\index.html");
            if (content.empty()) {
                sendResponse(clientSocket, 404, "text/plain",
"public/index.html not found");
            } else {
                sendResponse(clientSocket, 200, "text/html;
charset=utf-8", content);
            }
        } else if (url == "/style.css") {
            std::string content = readFileContent("public\\style.css");
            sendResponse(clientSocket, 200, "text/css", content);
        } else if (url == "/app.js") {
            std::string content = readFileContent("public\\app.js");
            sendResponse(clientSocket, 200, "application/javascript",
content);
        } else if (url == "/api/metrics") {
            sendResponse(clientSocket, 200, "application/json",
handleGetMetrics());
        } else if (url == "/api/vehicle/telemetry") {
            sendResponse(clientSocket, 200, "application/json",
handleGetVehicleTelemetry());
        } else if (url == "/api/ivi/apps") {
            sendResponse(clientSocket, 200, "application/json",
handleGetIviApps());
        } else if (url == "/api/processes") {
            sendResponse(clientSocket, 200, "application/json",
handleGetProcesses());
        } else if (url == "/api/priority/logs") {
```

```
            sendResponse(clientSocket, 200, "application/json",
prioController.getAuditLogsJson());
        } else if (url == "/api/db/dump") {
            sendResponse(clientSocket, 200, "text/plain",
dbManager.getLatestSqlDump());
        } else {
            sendResponse(clientSocket, 404, "text/plain", "404 Not
Found");
        }
    } else if (method == "POST") {
        if (url == "/api/process/priority") {
            sendResponse(clientSocket, 200, "application/json",
handlePostPriority(body));
        } else if (url == "/api/vehicle/gear") {
            sendResponse(clientSocket, 200, "application/json",
handlePostVehicleGear(body));
        } else if (url == "/api/schedule") {
            sendResponse(clientSocket, 200, "application/json",
handlePostSchedule(body));
        } else if (url == "/api/db/sync") {
            sendResponse(clientSocket, 200, "application/json",
handlePostDbSync());
        } else {
            sendResponse(clientSocket, 404, "application/json",
"{\"error\":\"Not Found\"}");
        }
    } else {
        sendResponse(clientSocket, 405, "text/plain", "Method Not
Allowed");
    }

    closesocket(clientSocket);
}

bool HttpServer::start() {
    WSADATA wsaData;
    if (WSAStartup(MAKEWORD(2, 2), &wsaData) != 0) {
        std::cerr << "[ERROR] WSAStartup failed: " << WSAGetLastError() <<
std::endl;
        return false;
```

```
    }

    serverSocket = socket(AF_INET, SOCK_STREAM, IPPROTO_TCP);
    if (serverSocket == INVALID_SOCKET) {
        std::cerr << "[ERROR] socket creation failed: " <<
WSAGetLastError() << std::endl;
        WSACleanup();
        return false;
    }

    // Allow port reuse
    int opt = 1;
    setsockopt(serverSocket, SOL_SOCKET, SO_REUSEADDR, (char*)&opt,
sizeof(opt));

    sockaddr_in serverAddr;
    serverAddr.sin_family = AF_INET;
    serverAddr.sin_addr.s_addr = INADDR_ANY;
    serverAddr.sin_port = htons(port);

    if (bind(serverSocket, (sockaddr*)&serverAddr, sizeof(serverAddr)) ==
SOCKET_ERROR) {
        std::cerr << "[ERROR] bind failed on port " << port << ": " <<
WSAGetLastError() << std::endl;
        closesocket(serverSocket);
        WSACleanup();
        return false;
    }

    if (listen(serverSocket, SOMAXCONN) == SOCKET_ERROR) {
        std::cerr << "[ERROR] listen failed: " << WSAGetLastError() <<
std::endl;
        closesocket(serverSocket);
        WSACleanup();
        return false;
    }

    isRunning = true;
    std::cout << "[HTTP] CarInfo Infotainment HUD Server listening on
http://localhost:" << port << std::endl;
```

```
    std::cout << "[HTTP] Ready to serve Car Cockpit Dashboard & Vehicle
APIs." << std::endl;

    while (isRunning) {
        sockaddr_in clientAddr;
        int clientLen = sizeof(clientAddr);
        SOCKET clientSocket = accept(serverSocket, (sockaddr*)&clientAddr,
&clientLen);

        if (clientSocket == INVALID_SOCKET) {
            if (!isRunning) break;
            continue;
        }

        ClientSession* session = new ClientSession{this, clientSocket};
        HANDLE hThread = CreateThread(NULL, 0, clientSessionWorker,
session, 0, NULL);
        if (hThread) {
            CloseHandle(hThread);
        } else {
            clientHandler(clientSocket);
            delete session;
        }
    }

    return true;
}

void HttpServer::stop() {
    if (isRunning) {
        isRunning = false;
        if (serverSocket != INVALID_SOCKET) {
            closesocket(serverSocket);
            serverSocket = INVALID_SOCKET;
        }
        WSACleanup();
        std::cout << "[HTTP] Server stopped." << std::endl;
    }
}
```

```
#ifndef HTTP_SERVER_H
```

```
#define HTTP_SERVER_H

#ifndef _WIN32_WINNT
#define _WIN32_WINNT 0x0600
#endif
#ifndef WINVER
#define WINVER 0x0600
#endif

#include <winsock2.h>
#include <ws2tcpip.h>
#include <string>
#include <atomic>
#include "monitor.h"
#include "priority_controller.h"
#include "scheduler.h"
#include "db_manager.h"

class HttpServer {
public:
    HttpServer(int port,
               SystemMonitor& monitor,
               PriorityController& prioController,
               CpuScheduler& scheduler,
               DatabaseManager& dbManager);
    ~HttpServer();

    bool start();
    void stop();
    void clientHandler(SOCKET clientSocket);

private:
    int port;
    SOCKET serverSocket;
    std::atomic<bool> isRunning;

    SystemMonitor& monitor;
    PriorityController& prioController;
    CpuScheduler& scheduler;
    DatabaseManager& dbManager;
```

```
    std::vector<SchedulingResult> lastSimulations;

    std::string readFileContent(const std::string& path);
    void sendResponse(SOCKET clientSocket, int statusCode, const
std::string& contentType, const std::string& body);

    // REST API handlers
    std::string handleGetMetrics();
    std::string handleGetProcesses();
    std::string handleGetIviApps();
    std::string handleGetVehicleTelemetry();
    std::string handlePostVehicleGear(const std::string& body);
    std::string handlePostPriority(const std::string& body);
    std::string handlePostSchedule(const std::string& body);
    std::string handlePostDbSync();
};

#endif // HTTP_SERVER_H
```